FLAT

UNIT - I Introduction to Finite Automata: Structural Representations, Automata and Complexity, the Central Concepts of Automata Theory – Alphabets, Strings, Languages, Problems.

Nondeterministic Finite Automata: Formal Definition, an application, Text Search, Finite Automata with Epsilon-Transitions.

Deterministic Finite Automata: Definition of DFA, How A DFA Process Strings, The language of DFA, Conversion of NFA with €-transitions to NFA without €-transitions. Conversion of NFA to DFA, Moore and Melay machines

UNIT - II Regular Expressions: Finite Automata and Regular Expressions, Applications of Regular Expressions, Algebraic Laws for Regular Expressions, Conversion of Finite Automata to Regular Expressions.

Pumping Lemma for Regular Languages, Statement of the pumping lemma, Applications of the Pumping Lemma.

Closure Properties of Regular Languages: Closure properties of Regular languages, Decision Properties of Regular Languages, Equivalence and Minimization of Automata.

UNIT - III Context-Free Grammars: Definition of Context-Free Grammars, Derivations Using a Grammar, Leftmost and Rightmost Derivations, the Language of a Grammar, Sentential Forms, Parse Tress, Applications of Context-Free Grammars, Ambiguity in Grammars and Languages.

Push Down Automata: Definition of the Pushdown Automaton, the Languages of a PDA, Equivalence of PDA's and CFG's, Acceptance by final state, Acceptance by empty stack, Deterministic Pushdown Automata. From CFG to PDA, From PDA to CFG.

UNIT - IV Normal Forms for Context- Free Grammars: Eliminating useless symbols, Eliminating €-Productions. Chomsky Normal form Griebech Normal form.

Pumping Lemma for Context-Free Languages: Statement of pumping lemma, Applications

Closure Properties of Context-Free Languages: Closure properties of CFL's, Decision Properties of CFL's

Turing Machines: Introduction to Turing Machine, Formal Description, Instantaneous description, The language of a Turing machine

UNIT - V Types of Turing machine: Turing machines and halting

Undecidability: Undecidability, A Language that is Not Recursively Enumerable, An Undecidable Problem That is RE, Undecidable Problems about Turing Machines, Recursive languages, Properties of recursive languages, Post's Correspondence Problem, Modified Post Correspondence problem, Other Undecidable Problems, Counter machines.

UNIT – 1

➢ Automata theory

Automata theory is the study of abstract machines and automata, as well as the computational problems that can be solved using them

Automata theory is the core of computer science. An automaton (plural: automata) is a self-operating machine. The abstract machine is called the automata.

Automatons are mathematical models for designing self-sufficient machines to solve predefined tasks

Automata theory is closely related to formal language theory. In this context, automata are used as finite representations of formal languages that may be infinite.

Automata theory is the basis for the theory of **formal languages**.

➢ CENTRAL CONCEPTS OF AUTOMATA THEORY

There are the basic terminologies that are important and frequently used in automata

Alphabet

An alphabet is a finite, non-empty set of symbols We use the symbol ∑ (sigma) to denote an alphabet

Example

a) ∑ = {a, b, c, d} is an alphabet set where 'a', 'b', 'c', and 'd' are symbols.
b) Binary: ∑ = {0,1}
c) All lower case letters: ∑ = {a,b,c,..z}
d) Alphanumeric: ∑ = {a-z, A-Z, 0-9}

String

A string is a finite sequence of symbols taken from ∑.

• Example − 'cabcad' is a valid string on the alphabet set ∑ = {a, b, c, d}

Empty string is donated by **ε (epsilon) or λ (lambda).**

Substring

Substring is a contiguous sequence of characters with in a string

Eg: let w = gate

Substrings = ε, g, a, t, e, ga, at, te, gat, ,ate, gate

Length of a String

It is the number of symbols/alphabets

Length of a string w, denoted by **|w|**.

E.g., if x = 010100 then |x| = 6

If |W|= 0, it is called an empty string (Denoted by λ or ε)

Concatenation of strings

Concatenation of strings is the technique to merging the symbols of strings together. Let A and B be two strings.

Then AB denotes the concatenation of A and B, that is, the string formed by making a copy of A and following it by a copy of B.

• If x = abc and y = de then xy = abcde

Concatenating the string A with empty strings i.e. 'ε' is string A only.

For example: εA = Aε = A.

Powers of an alphabet

Let ∑ be an alphabet. ∑k = the set of all strings of length k

• Examples

Let ∑ = {0,1}.

$\sum^0 = \{\varepsilon\}$ $\sum^1 = \{0,1\}$ $\sum^2 = \{00,01,10,11\}$

Kleene Closure / kleen Star / Kleen operator

• The Kleene star, ∑*, is a unary operator on a set of symbols or strings, ∑, that gives the infinite set of all possible strings of all possible lengths over ∑ including ε .

• Representation − ∑* = ∑0 U ∑1 U ∑2 U……. where ∑p is the set of all possible strings of length p

• Example − If ∑ = {a, b}, ∑* = { ε, a, b, aa, ab, ba, bb,…..... }

Kleene Closure Plus / Positive closure

• The set ∑+ is the infinite set of all possible strings of all possible lengths over ∑ excluding ε.

• Representation $\sum^+ = \sum 1 \cup \sum 2 \cup \sum 3 \cup \ldots\ldots$

$\sum^+ = \sum^* - \{\varepsilon\}$

• Example − If ∑ = { a, b } , $\sum^+$ = { a, b, aa, ab, ba, bb,… .....}

Language

A language is a subset of ∑* for some alphabet ∑.

It can be finite or infinite.

• If ∑ is an alphabet, and L ⊆ ∑* , then L is a language over ∑.

• A language over ∑ may not include strings with all symbols of ∑.

Example :

The language of all strings consisting of n 0's followed by n 1' for some n ≥ 0:

{ε, 01, 0011, 000111, …}

– ∑* is a language

– Empty set is a language. The empty language is denoted by Φ

– The set {ε} is a language, {ε} is not equal to the empty language.

– The set of all identifiers in a programming language is a language.

– The set of all syntactically correct C programs is a language.

Example –

1. If the language takes all possible strings of length 2 over ∑ = {a, b}, then
   L = { ab, bb, ba, bb}
2. Let L be the language of all strings consisting of n 0's followed by n 1's: then
   L = {01,0011,000111,…}

➢ **Set-Formers to Define Languages**

A set-former is a mathematical notation used to define a formal language by specifying the properties that strings over an alphabet must satisfy.

• A set-former is a common way to define a language

Set-former: {w | something about w}

Ex: {w | w consists of equal number of 0's and 1's}
{w | w is a binary integer that is prime}

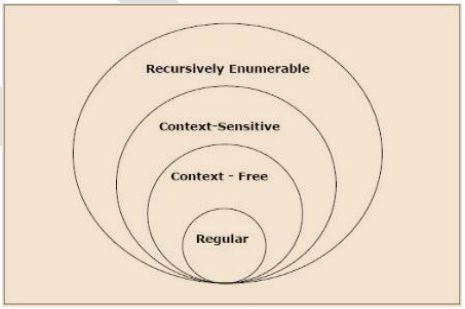
Sometimes we replace w with an expression
$\{ 0^n1^n \mid n \geq 1 \}$
$\{ 0^i1^j \mid 0 \leq i \leq j \}$

- **Language - Problem**
  • In automata theory, a problem is the question of deciding whether a given string is a member of a particular language.
  • If $\sum$ is an alphabet, and L is a language over $\sum$, then the problem L is: Given a string w in $\sum^*$, decide whether or not w is in L.
  • In order to make decision requires some computational resources.
  – Deciding whether a given string is a correct C identifier
  – Deciding whether a given string is a syntactically correct C program.
  • Some decision problems are simple, some others are harder.
  • A decision question may require exponential resources in the size of its input.
  • A decision question may be unsolvable.

There are several types of automata, including:

- **Finite automata**

A simple type of automaton that can recognize patterns in input sequences. Finite automata are used in text processing, compilers, and network protocols.



- **Pushdown automata**

Used for syntax analysis in compilers, evaluating arithmetic expressions, and more. Pushdown automata use a stack to manage nested elements.

- **Linear-bounded automata**

Used for implementing genetic programming and recognizing context-sensitive languages.

- **Turing machines**

The most powerful automata, used for solving recursively enumerable problems and understanding complexity theory.

This is a hierarchy therefore every language of type 3 is also of type 2, 1 and 0. Similarly every language of type 2 is also of type 1 and 0 etc.

- **FINITE AUTOMATA OR FINITE STATE MACHINE**
  - In **Finite Automata, Finite** means a finite number of states and Automata means the **Automatic** Machine which works without any interference of human beings.
  - An automaton with a finite number of states is called a finite automaton (FA) or finite-state machine (FSM).
  - Finite automata are abstract machines used to recognize patterns
  - It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.

- At the time of transition, the automata can either move to the next state or stay in the same state.
- A state is a representation of the current status of an automaton that captures essential information about the input processed so far.
- Finite automata have two states, **Accept state(final state)** or **Reject state**. When the input string is processed successfully, and the automata reached its final state, then it will accept otherwise, it is rejected.
- They are widely used in text processing, compilers, and network protocols.

Applications Of Finite Automata:

- Automata theory presents many useful models for software and hardware.
- In compilers we use finite automatons for lexical analyzers, and push down automatons for parsers.
- In search engines, we use finite automatons in order to determine tokens in web pages.
- A Finite Automata is highly useful in designing text editors.
- A Finite Automata is highly useful in designing spell checkers
- Finite automata model protocols, electronic circuits.
  Context-free grammars are used to describe the syntax of essentially every programming language.
- Automata theory offers many useful models for natural language processing

➢ **Finite Automata Model:**

Finite automata can be represented by input tape, read head and finite control.

Input tape: It is a linear tape having some number of cells. Each input symbol is placed in each cell.

Finite control: The finite control decides the next state on receiving particular input from input tape.

Tape Reader / Read Head : The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

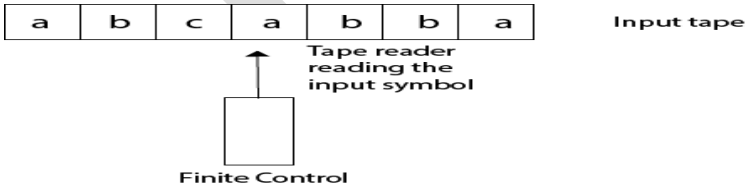


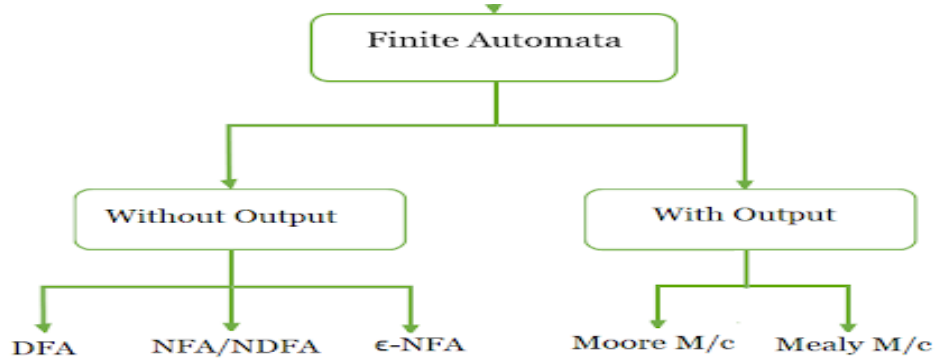
Fig :- Finite automata model

Formal Definition of FA

A finite automaton is a collection of 5-tuple (Q, ∑, δ, q0, F), where:

1. Q: finite set of states
2. ∑: finite set of the input symbol
3. q0: initial state
4. F: final state or accept state
5. δ: Transition function

Types of Finite Automata



There are two types of finite automata:

1. Deterministic Fintie Automata (DFA)
2. Non-Deterministic Finite Automata (NFA)

1. Deterministic Finite Automata (DFA)

A DFA is represented as {Q, Σ, q, F, δ}. In DFA, for each input symbol, the machine transitions to one and only one state. DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

Formal Definition of a DFA

A DFA can be represented by a 5-tuple (Q, Σ, δ, q0, F) where −

Q is a finite set of states.

∑ is a finite set of symbols called the alphabet.

δ is the transition function where δ: Q × ∑ → Q

q0 is the initial state from where any input is processed (q0 Q).

F is a set of final state/states of Q (F ⊆ Q).

A transition function that takes as arguments a state and an input symbol and returns a state The transition function will commonly be denoted **δ**.

In our informal graph representation of automata was represented by arcs between states and the labels on the arcs If q is a state and a is an input symbol then δ(q,a) is that state p such that there is an arc labelled a from q to p

2. Non-Deterministic Finite Automata (NFA)

In NDFA, for a particular input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called Non-deterministic Automaton. As it has finite number of states, the machine is called Non-deterministic Finite Machine or Non-deterministic Finite Automaton.

Formal Definition of an NDFA / NFA

An NDFA can be represented by a 5-tuple (Q, Σ, δ, q0, F) where −

Q is a finite set of states.

∑ is a finite set of symbols called the alphabets.

δ is the transition function where δ: Q × ∑ → 2^Q

(Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)
q0 is the initial state from where any input is processed (q_0 Q)
F is a set of final state/states of Q (F ⊆ Q)

NFA is similar to DFA but includes the following features:

- It can transition to multiple states for the same input.
- It allows null (ϵ) moves, where the machine can change states without consuming any input.
- Every DFA is NFA, but NFA is not DFA.
- There can be multiple final states in both NFA and DFA.
- DFA is used in Lexical Analysis in Compiler.
- NFA is more of a theoretical concept.

➢ **Simpler Notations for DFA**

There are two preferred notations for describing automata

1. A transition diagram which is a graph
2. A transition table which is a tabular listing of the function which by implication tells us the set of states and the input alphabet

1.Transition Diagram

A transition diagram for a DFA A =(Q, ∑, δ, q0, F) is a graph defined as follows

- For each state in Q there is a node
- For each state q in Q and each input symbol a in ∑ let $\delta(q,a)=p$. Then the transition diagram has an arc from node q to node p labeled a. If there are several input symbols that cause transitions from q to p then the transition diagram can have one arc labeled by the list of these symbols
- There is an arrow into the start state q0 labeled Start. This arrow does not originate at any node
- Nodes corresponding to accepting states (those in F) are marked by a double circle. States not in F have a single circle

Ex:

DFA with ∑ = {0, 1} accepts all strings starting with 1.

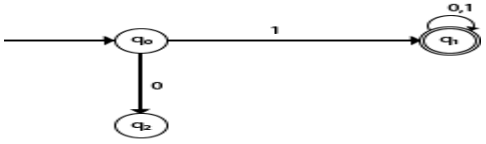


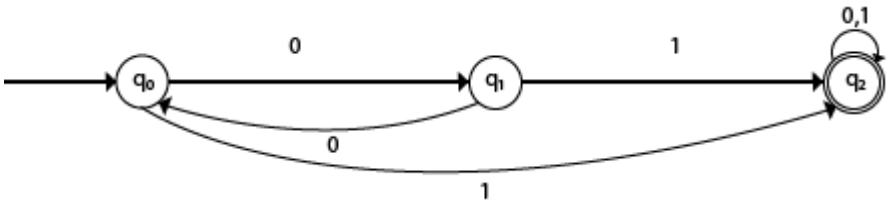
Fig: Transition diagram

2. Transition Table

A transition table is a conventional tabular representation of a function like δ that takes two arguments and returns a value. The rows of the table correspond to the states and the columns correspond to the inputs The entry for the row corresponding to state **q** and the column corresponding to input **a** is the $\delta(q,a)$

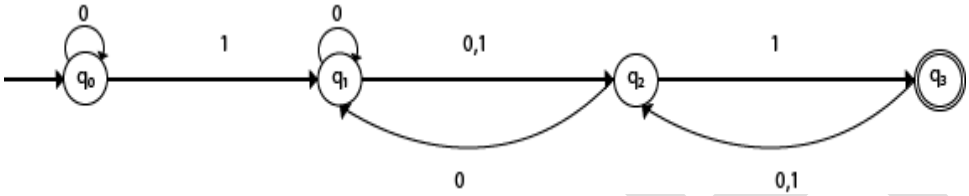
The start state is marked with an arrow and the accepting states are marked with a star.

Ex ;



Present State	Next state for Input 0	Next State of Input 1
→q0	q1	q2
q1	q0	q2
*q2	q2	q2

Ex:



Present State	Next state for Input 0	Next State of Input 1
→q0	q0	q1
q1	q1, q2	q2
q2	q1	q3
*q3	q2	q2

Ex: Let us formally specify a DFA that accepts all and only the strings of 0's and 1's that have the sequence 01 somewhere in the string

Solution :

We can write this language L as

{w | w is of the form x01y for some strings x and y consisting of 0's and 1's only}

Another equivalent description using parameters x and y to the left of the vertical bar i s

{x01y | x and y are any strings of 0's and 1's}

Examples of strings in the language include 01,11010 and 100011 etc.

Examples of strings not in the language ε, 0, 111000, …

Substring of language is 01

Minimum length of substring is |01| = 2

No. of states = min. substring length + 1 = 3 states Q= {q0, q1, q2}

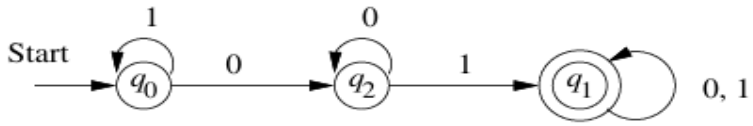
Input Alphabet is ∑ = {0,1}

Initial state is q0 and Final state is {q1}

- Has it already seen 01 . If so then it accepts every sequence of further inputs ie it will only be in accepting states from now on
- Has it never seen 01, but its most recent input was 0 so if it now sees a 1 it will have seen 01 and can accept everything it sees from here on
- Has it never seen 01 but its last input was either non-existent (it just started) or it last saw a 1. In this case A cannot accept until it first sees a 0 and then sees a 1 immediately after

δ(q0, 1) = q0
δ(q0, 0) = q2
δ(q2,0) = q2
δ(q2, 1)= q1
δ(q1,0) = δ(q1, 1) = q1
so, the formal DFA A = ({q0, q1, q2}, {0,1}, δ, q0, {q1})

	0	1
→ q_0	q_2	q_0
*q_1	q_1	q_1
q_2	q_2	q_1



- **Transition function:**

  In automata theory, the transition function is a well-defined function that tells about the movement from one state to another upon receiving an input symbol.
  It is writen in set form as δ : Q X Σ --> Q.
  It is described by the standard syntax: δ(q0, a)- → q1
  where
  q0 is the current state
  q1 is the output state
  a is the input symbol
  This transition function describes that the FA machine should move from state q0 to q1 upon receiving a symbol while being present at state q0. Thus it is called transition function because of movement or transition from one state to another

- **Extended transition function**

  Extended transition function that describes what happens when we start in any state and follow any sequence of inputs.
  If δ is our transition function then the extended transition function constructed from δ will be called $\hat{\delta}$ (Delta cap)
  The extended transition function is defined recursively.
  An extended transition function takes two arguments. The first argument is a state q and the second argument is a string w
  It is extended to work on an input string w rather than a single input symbol a.
  A transition function performs a transition on a *single* character, while an extended transition function performs a transition on a *complete* string.

- **Extended transition function for DFA**

Transition Function δ : Q X Σ -→> Q.
Extended Transition Function $\hat{\delta}$: Q X Σ* = Q
Basis : $\hat{\delta}$ (q, ε) = q if no input on state q , results q only.
Induction : $\hat{\delta}$ (q,w) = δ ($\hat{\delta}$ (q, x), a)
Suppose **w** is a string of the form **xa** that is **a** is the last symbol of **w** and **x** is the string consisting of all but the last symbol
For example w= 1101 is broken into x=110 and a= 1

To compute δˆ(q, w), first compute $\hat{\delta}$ (q, x), the state that the automaton is in after processing all but the last symbol of w $\hat{\delta}$

Suppose this state is p, i.e., $\hat{\delta}$ (q, x) = p

Then $\hat{\delta}$ (q, w) is what we get by making a transition from state p on input a - the last symbol of w

➢ **Language of DFA**

The language accepted by a dfa M = (Q,∑, δ, q0, F) is the set of all strings on ∑ accepted by M. In formal notation, $L(M) = \{w \mid \hat{\delta}(q0, w) \in F\}$.

Example : Design a DFA to accept the language
L= {w | w is of even length and begins with 01 }

Solution:

The automaton needs to remember whether the string seen so far started with 01. It also needs to keep track of the length of the string. Hence it contains five states and they are:

q0: The initial state.

q1: The state entered on reading 0 in state q0.

q2: The state entered on reading 01 initially. The automation subsequently returns to this state whenever the substring seen so far starts with 01 and is of even length.

q3: The DFA enters this state whenever the substring seen so far starts with 01 and is of odd length.

q4: This state is encountered whenever a 1 is encountered in state q0 or a 0 is encountered in state q1.

q2 is the only accepting state.

The DFA can be given as

M= ({q0, q1, q2, q3, q4},{0,1}, δ , q0 , {q2}) .

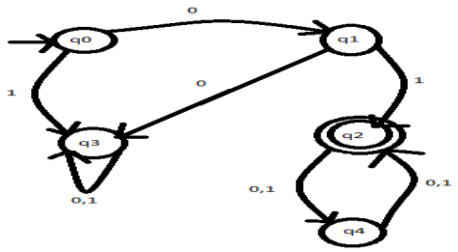
where δ the transition function is given by the transition diagram

Representation of this DFA in the form of transition diagram.

Check whether the string 011101 is accepted by DFA or not.

Since this string starts with 01 and is of even length it is in the language.

Thus, we expect that $\hat{\delta}$ (q0 , 011101) = q2 . Since q2 is the only accepting state.



The check involves computing $\hat{\delta}$ (q0,w) for each prefix w of 011101,starting at ε and going in increasing size. (ε011101 start from left to right)

$\hat{\delta}(q0,\varepsilon) = q0.$

$\hat{\delta}(q0, \varepsilon0) = \delta(\hat{\delta}(q0,\varepsilon),0) = \delta(q0,0) = q1.$

$\hat{\delta}(q0,01) = \delta(\hat{\delta}(q0,0),1) = \delta(q1,1) = q2.$

$\hat{\delta}(q0,011) = \delta(\hat{\delta}(q0,01),1) = \delta(q2,1) = q3.$

$\hat{\delta}(q0,0111) = \delta(\hat{\delta}(q0,011),1) = \delta(q3,1) = q2.$

$\hat{\delta}(q0,01110) = \delta(\hat{\delta}(q0,0111),0) = \delta(q2,0) = q3.$

$\hat{\delta}(q0,011101) = \delta(\hat{\delta}(q0,01110),1) = \delta(q3,1) = q2.$

Reached final state so string is acceptable

Example: Let us design a DFA to accept the language L
{w | w has both an even number of 0's and an even number of 1's}

Solution

The state is used to remember whether the number of s seen so far is even or odd and also to remember whether the number of s seen so far is even or odd There are thus four states which can be given the following interpretations

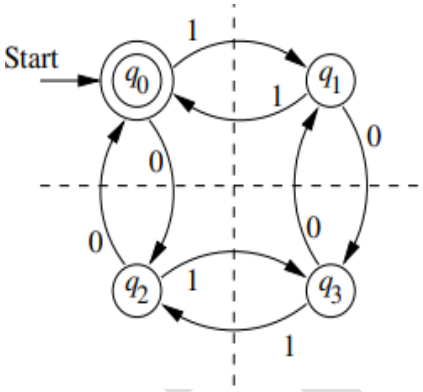
q0 : Both the number of 0's seen so far and the number of 1's seen so far are even

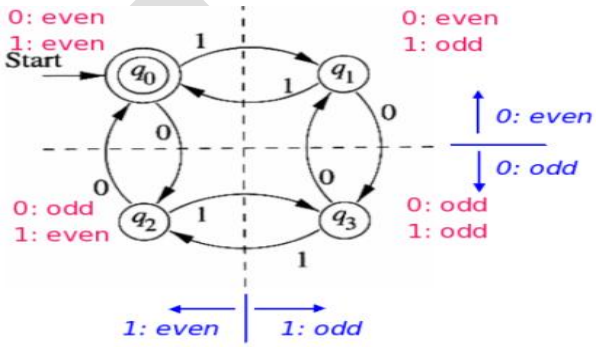
q1: The number of 0's seen so far is even but the number of 1's seen so far is odd

q2: The number of 1's seen so far is even but the number of 0's seen so far is odd

q3: Both the number of 0's seen so far and the number of 1's seen so far are oddg

State q0 is both the start state and the lone accepting state It is the start state because before reading any inputs the numbers of 0's and 1's seen so far are both zero and zero is even It is the only accepting state because it describes exactly the condition for a sequence of 0's and 1's to be in L

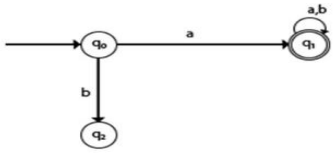




L= ({q0,q1,q1,q2}, {0,1}, δ , q0, {q0})

- **Dead state**

If a transition goes to a state from which it can never escape. Such a state is called a trap state/dead state/sink. Dead state is also known as dummy state.



In the above example, q2 is a trap or dead state because it can't reach the final state.

A dead state in a deterministic finite automaton (DFA) is a state that cannot reach any final state. It is non-final state. Transitions for all inputs on dead state are taken to it self only.

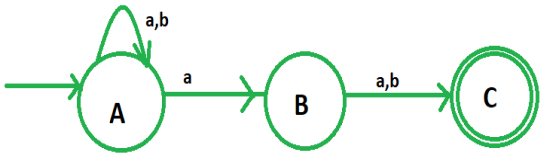
- Dead state may be required in DFA.
- Dead state is not required in NFA.

When the automaton encounters an input symbol that is not defined for the current state, it should transition to a dead state. This is crucial for ensuring that the automaton can handle unexpected or invalid inputs

When constructing a deterministic finite automaton (DFA), every state must have a defined transition for every possible input symbol in the alphabet. If a state does not have a defined transition for an input symbol, adding a dead state ensures that the automaton remains deterministic.

➢ **Nondeterministic Finite Automata**

Example 1 The NFA of the language containing all the strings in which 2nd symbol from the RHS is "a" is: ∑ = {a, b}

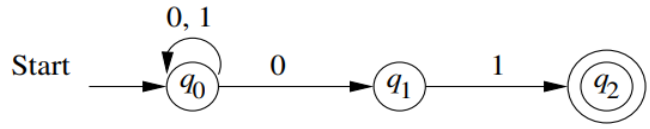


NFA

	a	b
→A	{A,B}	{A}
B	{C}	{C}
©	0	0

Example 2

A nondeterministic finite automaton whose job is to accept all and only the strings of 0's and 1's that ends with 01



The NFA of Fig can be specified formally as ({q0, q1, a2}, {0,1}, δ, q0, {q2})

	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	∅	$\{q_2\}$
$*q_2$	∅	∅

When there is no transition at all from a given state on a given input symbol the proper entry is ∅. The only difference is that each entry in the table for the NFA is a set even if the set has one member

Extended Transition Function for NFA

Extended transition function of an NFA to a function $\hat{\delta}$ that takes a state q and a string of input symbols w and returns the set of states that the NFA is in if it starts in state q and processes the string w

Transition function of NFA is $\delta: Q \times \sum \rightarrow 2^Q$

The **extended transition function** $\hat{\delta}$ extends the normal transition function δ to process **entire strings** rather than single symbols. It is defined recursively:

- $\hat{\delta}$ (q, ε) = {q} ((Base case: Processing empty string stays in the same state)

Suppose w is of the form w = xa where a is the final symbol of w and x is the rest of w.

$\hat{\delta}$ (q, w) = $\hat{\delta}$ (q, xa)

= δ($\hat{\delta}$ (q, x) , a)

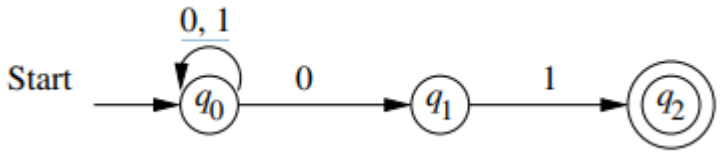
Let $\hat{\delta}$ (q, x) = {p1, p2, ….pk }

= δ(p1, a) U δ(p2, a) U ….. δ (pk, a)

$\hat{\delta}$ (q, w) = $\bigcup_{i=1}^{n} \delta(pi, a)$ = { r1, r2, … rk} resulting states

The language L accepted by an NFA M = (Q,∑, δ, q0,F) is defined as the set of all strings accepted in the above sense. Formally**, L (M)= {w | $\hat{\delta}$ (q0, w)⋂ F≠Ø} .**

Ex : use $\hat{\delta}$ to describe how the string 00101 is processed by the NFA



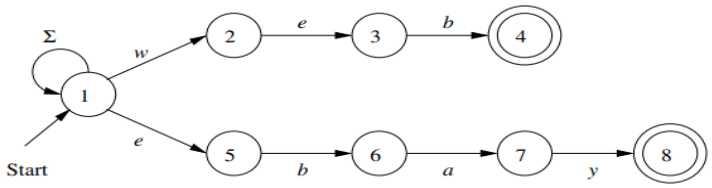
1. $\hat{\delta}(q_0, \epsilon) = \{q_0\}$.
2. $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$.
3. $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
4. $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.
5. $\hat{\delta}(q_0, 0010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
6. $\hat{\delta}(q_0, 00101) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.

- **DFA vs NFA**

DFA	**NFA**
DFA stands for Deterministic Finite Automata.	NFA stands for Nondeterministic Finite Automata.
For each symbolic representation of the alphabet, there is only one state transition in DFA.	No need to specify how does the NFA react according to some symbol.
DFA cannot use Empty String transition.	NFA can use Empty String transition.
DFA can be understood as one machine.	NFA can be understood as multiple little machines computing at the same time.
In DFA, the next possible state is distinctly set.	In NFA, each pair of state and input symbol can have many possible next states.
DFA is more difficult to construct.	NFA is easier to construct.

DFA	NFA
DFA rejects the string in case it terminates in a state that is different from the accepting state.	NFA rejects the string in the event of all branches dying or refusing the string.
Time needed for executing an input string is less.	Time needed for executing an input string is more.
All DFA are NFA.	Not all NFA are DFA.
DFA requires more space.	NFA requires less space then DFA.
Dead configuration is not allowed. eg: if we give input as 0 on q0 state so we must give 1 as input to q0 as self loop.	Dead configuration is allowed. eg: if we give input as 0 on q0 state so we can give next input 1 on q1 which will go to next state.
δ: Q X Σ -> Q i.e. next possible state belongs to Q.	For NFA δ: Q X Σ → 2^Q i.e. next possible state belongs to power set of Q. For ϵ-NFA $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
Conversion of Regular expression to DFA is difficult.	Conversion of Regular expression to NFA is simpler compared to DFA.
Epsilon move is not allowed in DFA	Epsilon move is allowed in NFA
In a DFA, there is only **one** possible transition for each input symbol from any given state.	In an NFA, there can be **multiple** transitions for a single input symbol from given state
$\hat{\delta}$ (q,w) = δ($\hat{\delta}$ (q, x), a) = p	$\hat{\delta}$ (q, w) = = $\hat{\delta}$ (q, xa) = δ($\hat{\delta}$ (q, x) , a)= $\bigcup_{i=1}^{n} \delta(pi, a)$ = { r1, r2, … rk} resulting states

Example : NFA to recognise words '*web'* and '*ebay'*



∑ → set of all printable ASCII characters

➢ **CONVERTING NFA TO DFA (Or) EQUIVALENCE OF DFA AND NFA**

- Each state in the DFA represents a **set of states** (S) in the NFA.
- The number of states in the resulting DFA may differ from that of the original NFA.
- If the NFA has **N states**, the DFA can have at most $\mathbf{2^N}$ **states**.
- If a DFA state represents a set of NFA states, the transition for a symbol follows **all Possible transitions** from all those states.
- Every nondeterministic finite automaton (NFA) can be converted to a deterministic finite automaton (DFA).
- Given an NFA

$$N = (Q_N, \Sigma, \delta_N, q0, F_N)$$

- we will construct a DFA

$$D = (Q_D, \Sigma, \delta_D, \{q0\}, F_D)$$
$$\text{such that } L(D) = L(N)$$

- Notice that the input alphabets of the two automata are the same and the start state of D is the set containing only the start state of N

The following steps are followed to convert a given NFA to a DFA-

Step-01:

Let Q' be a new set of states of the DFA. Q' is null in the starting.

Let T' be a new transition table of the DFA.

Step-02:

Add start state of the NFA to Q' i.e. Initial state in DFA = Initial state in NFA

- Add transitions of the start state to the transition table T'.
- If start state makes transition to multiple states for some input alphabet, then treat those multiple states as a single state in the DFA.

Step-03:

If any new state is present in the transition table T',

- Add the new state in Q'.
- Add transitions of that state in the transition table T' using

$$\delta_D(S, a) = \bigcup_{p \text{ in } S} \delta_N(p, a)$$

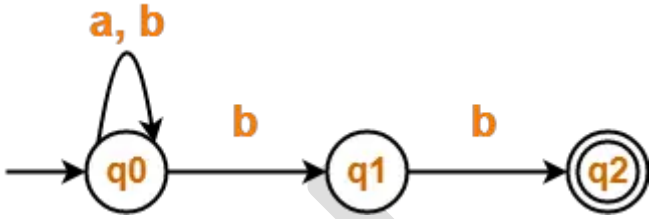
Step-04:

Keep repeating Step-03 until no new state is present in the transition table T'.

Finally, the transition table T' so obtained is the complete transition table of the required DFA.
If "x" was the final state in NFA, then all those states will be the final in DFA where "x" exists.
It means,
$\mathbf{S \cap F_N \neq \emptyset}$ where S is set of subsets of Q_N and S is one of state in DFA

Example :

Convert the following Non-Deterministic Finite Automata (NFA) to Deterministic Finite Automata (DFA)- NFA for String ending with 'bb'



Solution

Transition table for the given Non-Deterministic Finite Automata (NFA) is-

State / Alphabet	**A**	**B**
→q0	q0	q0, q1
q1	**Ø**	*q2
***q2**	**Ø**	**Ø**

Step-01:

Let Q' be a new set of states of the Deterministic Finite Automata (DFA).
Let T' be a new transition table of the DFA.

Step-02:

Add transitions of start state q0 to the transition table T'.

State / Alphabet	**A**	**B**
→q0	q0	{q0, q1}

Step-03:

New state present in state Q' is {q0, q1}.
Add transitions for set of states {q0, q1} to the transition table T'.

State / Alphabet	**A**	**B**
→q0	q0	{q0, q1}
{q0, q1}	q0	{q0, q1, q2}

Step-04:

New state present in state Q' is {q0, q1, q2}.
Add transitions for set of states {q0, q1, q2} to the transition table T'.

State / Alphabet	**A**	**B**
→q0	q0	{q0, q1}

{q0, q1}	q0	{q0, q1, q2}
{q0, q1, q2}	q0	{q0, q1, q2}

Step-05:

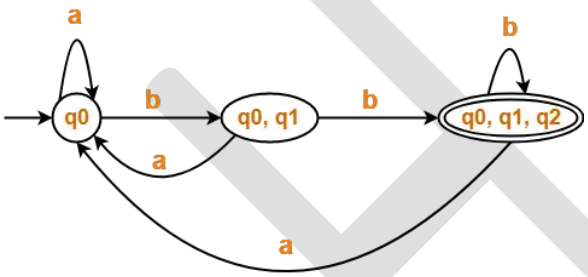
Since no new states are left to be added in the transition table T', so we stop.

States containing q2 as its component are treated as final states of the DFA.

Finally, Transition table for Deterministic Finite Automata (DFA) is-

State / Alphabet	**A**	**B**
→q0	q0	{q0, q1}
{q0, q1}	q0	*{q0, q1, q2}
***{q0, q1, q2}**	q0	*{q0, q1, q2}

Now, Deterministic Finite Automata (DFA) may be drawn as-



Deterministic Finite Automata (DFA)

- **Epsilon Transitions**

  In finite state automata, epsilon transitions allow a machine to move from one state to another without consuming any input symbol, essentially representing a "null" move

- **ε-NFA**

  An **epsilon nondeterministic finite automaton (NFA)** has null or epsilon transitions from one state to another. Epsilon NFA is also called a **null NFA** or an **NFA lambda**.

  A regular expression for a language forms an epsilon NFA. This epsilon NFA then converts to a simple NFA. We then use the simple NFA to make a deterministic finite automaton (DFA).

  Formal notation for ε-NFA

  Formally we represent an ε-NFA A by `A = =(Q, Σ, δ, q0, F)` where all components have their same interpretation as for an NFA except that δ is now a function that takes as arguments

  Q is a finite set of states
  - Σ is a finite set of symbols
  - $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$ is the transition function
  - $q0 \in Q$ is the initial state
  - $F \subseteq Q$ is the set of final states

- **Extended Transition function for ε-NFA**

Extended transition function of an ε-NFA to a function $\hat{\delta}$ that takes a state q and a string of input symbols w and returns the set of states that the ε-NFA is in if it starts in state q and processes the string w

Transition function of ε-NFA is $\delta: Q \times (\sum \cup \varepsilon) \rightarrow 2^Q$

Extended Transition function of ε-NFA is $\hat{\delta} : Q \times \sum^* \rightarrow 2^Q$

The **extended transition function** $\delta^\wedge$ extends the normal transition function δ to process **entire strings** rather than single symbols. It is defined recursively:

- $\hat{\delta}(q, \varepsilon) = $ E-closure(q)

  (Base case: Processing empty string stays in the same state)

INDUCTION

Suppose w is of the form w = xa where a is the final symbol of w and x is the rest of w.

$\hat{\delta}(q, w) = \hat{\delta}(q, xa)$

$= \delta(\hat{\delta}(q, x), a)$

Let $\hat{\delta}(q, x) = \{p1, p2, \ldots.pk\}$

$= \delta(p1, a) \cup \delta(p2, a) \cup \ldots.. \delta(pk, a)$

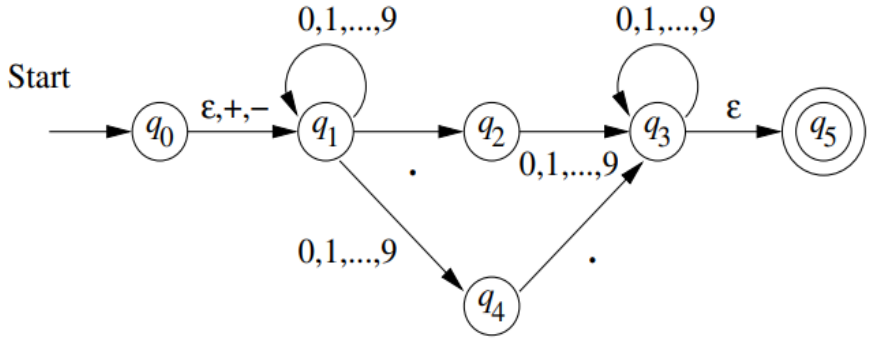
$\hat{\delta}(q, w) = \bigcup_{i=1}^{n} \delta(p, a) = \bigcup_{j}^{k} E - closure(rj)$

let { r1, r2, … rk} resulting states

The language L accepted by an epsilon- NFA M = (Q,∑, δ, q0,F) is defined as the set of all strings accepted in the above sense. Formally**, L (M)= {w | δ* (q0, w)⋂ F≠Ø} .**

Example

ε−NFA that accepts decimal numbers consisting of
An optional + or − sign
A string of digits
A decimal point and
Another string of digits. Either this string of digits or the string 2 can be empty but at least one of the two strings of digits must be non empty.



Formally represented as

Transition table is

$$E = (\{q_0, q_1, \ldots, q_5\}, \{., +, -, 0, 1, \ldots, 9\}, \delta, q_0, \{q_5\})$$

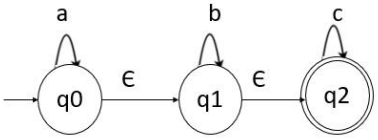
	ϵ	$+, -$	$.$	$0, 1, \ldots, 9$
q_0	$\{q_1\}$	$\{q_1\}$	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$	$\{q_1, q_4\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_3	$\{q_5\}$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_4	$\emptyset$	$\emptyset$	$\{q_3\}$	$\emptyset$
q_5	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

- **Epsilon-Closures**

  The ε closure(P) is a set of states which are reachable from state P on ε-transitions including the state itself..

  Example

  Find ε-closure for the following Non-deterministic finite automata (NFA) with epsilon.



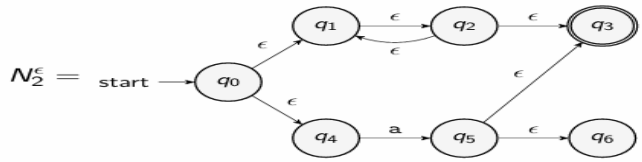
  ε-closure (q0)= {q0,q1,q2} self state+ ε-reachable states.

  ε-closure (q1)= { q1,q2}

  q1 is self-state and q2 is a state obtained from q1 with epsilon input.

  ε-closure (q2)= {q2}

Example



$$\begin{aligned}
ECl_o(q_0) &= \{q_0, q_1, q_2, q_3, q_4\} \\
ECl_o(q_2) &= \{q_1, q_2, q_3\} \\
ECl_o(q_5) &= \{q_3, q_5, q_6\} \\
ECl_o(\{q_2, q_5\}) &= ECl_o(q_2) \cup ECl_o(q_5) \\
&= \{q_1, q_2, q_3\} \cup \{q_3, q_5, q_6\} \\
&= \{q_1, q_2, q_3, q_5, q_6\}
\end{aligned}$$

- **CONVERTING NFA WITH ε TRANSITIONS TO NFA WITHOUT ε TRANSITIONS**
- ε-NFA → NFA

  Let, M = (Q, Σ, δ, q_0, F) be an NFA with ε transitions.

  Construct M' = (Q, Σ, δ', q_0, F') M' is a NFA without ε moves.

  1. Find out all the ε transitions from each state from Q. That will be called as ε-closure {q} where $q_i \epsilon$ Q.

  2. Then δ' transitions can be obtained. The δ' transitions means an ε-closure on δ moves.

  3. Step-2 is repeated for each input symbol and for each state of given NFA.

4. Using the resultant states the transition table for equivalent NFA without & can be built.

5. Consider the states which can be accessible to final state of epsilon NFA with epsilon input i.e. If any state in the **ε-closure** of a given state is a final state, the given state itself becomes a final state.

Rule for conversion

$\delta'(q, a) = \varepsilon\text{-closure}(\delta(\delta(q, \varepsilon), a))$

where $\delta(q,\varepsilon) = \varepsilon$ - closure (q) , so

$\delta'(q, a) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q), a))$

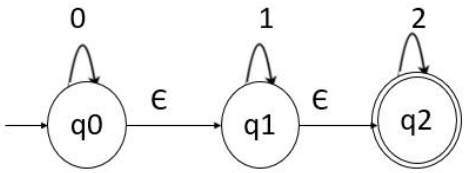
No change in Initial state

No change in total number of states

May be change in final state

Changes in transition.

Example : Convert the given NFA with epsilon to NFA without epsilon.



We will first obtain ε-closure of each state i.e., we will find ε-reachable states from the current state.

- ε-closure(q0) = {q0, q1, q2}
- ε-closure(q1) = {q1, q2}
- ε-closure(q2) = {q2}

Now we will obtain *δ'* transitions for each state on each input symbol as shown below –

$\delta'(q0, 0) = \varepsilon\text{-closure}(\delta(\delta\hat{}(q0, \varepsilon),0))$
$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q0),0))$
$= \varepsilon\text{-closure}(\delta(q0,q1,q2), 0))$
$= \varepsilon\text{-closure}(\delta(q0, 0) \cup \delta(q1, 0) \cup \delta(q2, 0))$
$= \varepsilon\text{-closure}(q0 \cup \Phi \cup \Phi)$
$= \varepsilon\text{-closure}(q0)$
$= \{q0,q1, q2\}$

$\delta'(q0, 1) = \varepsilon\text{-closure}(\delta(\delta\hat{}(q0, \varepsilon),1))$
$= \varepsilon\text{-closure}(\delta(q0,q1,q2), 1))$
$= \varepsilon\text{-closure}(\delta(q0, 1) \cup \delta(q1, 1) \cup \delta(q2, 1))$
$= \varepsilon\text{-closure}(\Phi \cup q1 \cup \Phi)$
$= \varepsilon\text{-closure}(q1)$
$= \{q1, q2\}$

$\delta'(q0, 2) = \varepsilon\text{-closure}(\delta(\delta\hat{}(q0, \varepsilon),2))$
$= \varepsilon\text{-closure}(\delta(q0,q1,q2), 2))$
$= \varepsilon\text{-closure}(\delta(q0, 2) \cup \delta(q1, 2) \cup \delta(q2, 2))$
$= \varepsilon\text{-closure}(\Phi \cup \Phi U q2)$
$= \varepsilon\text{-closure}(q2)$
$= \{q2\}$

δ'(q1, 0) = ε−closure(δ(δ^(q1, ε),0))
= ε−closure(δ(q1,q2), 0))
= ε−closure(δ(q1, 0) U δ(q2, 0))
= ε−closure(ϕ ∪ ϕ)
= ε−closure(ϕ)
= ϕ

δ'(q1,1) = ε−closure(δ(δ^(q1, ε),1))
= ε−closure(δ(q1,q2), 1))
= ε−closure(δ(q1, 1) U δ(q2, 1))
= ε−closure(q1 ∪ ϕ)
= ε−closure(q1)
= {q1,q2}

δ'(q1, 2) = ε−closure(δ(δ^(q1, ε),2))
= ε−closure(δ(q1,q2), 2))
= ε−closure(δ(q1, 2) U δ(q2, 2))
= ε−closure(ϕ ∪ q2)
= ε−closure(q2)
= {q2}

δ'(q2, 0) = ε−closure(δ(δ^(q2, ε),0))
= ε−closure(δ(q2), 0))
= ε−closure(δ(q2, 0))
= ε−closure(ϕ)
= ϕ

δ'(q2, 1) = ε−closure(δ(δ^(q2, ε),1))
= ε−closure(δ(q2), 1)
= ε−closure(δ(q2, 1))
= ε−closure(ϕ)
= ϕ

δ'(q2, 2) = ε−closure(δ(δ^(q2, ε),2))
= ε−closure(δ(q2), 2))
= ε−closure(δ(q2, 2))
= ε−closure(q2)
= {q2}

Now, we will summarize all the computed δ' transitions as given below −

δ'(q0,0)={q0,q1,q2}
δ'(q0,1)={q1,q2}
δ'(q0,2)={q2}

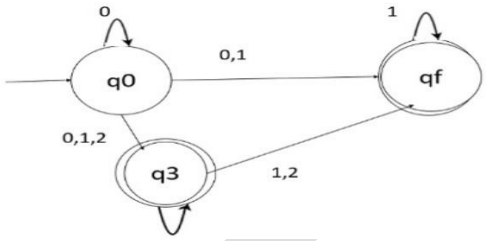
δ'(q1,0)= { Φ }
δ'(q1,1)={q1,q2}
δ'(q1,2)={q2}

δ'(q2,0)={ Φ }
δ'(q2,1)={ Φ }
δ'(q2,2)={q2}

The **transition table** is given below −

States\inputs	0	1	2
q0	{q0,q1,q2}	{q1,q2}	{q2}
q1	Φ	{q1,q2}	{q2}
q2	Φ	Φ	{q2}

The NFA without epsilon is given below −



Steps for converting NFA with ε to DFA (ε-NFA to DFA)

Step 1: We will take the ε-closure for the starting state of NFA as a starting state of DFA.

Step 2: Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

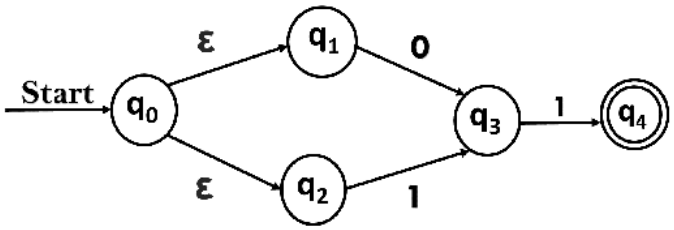
Step 3: If we found a new state, take it as current state and repeat step 2.

Step 4: Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

Step 5: Mark the states of DFA as a final state which contains the final state of NFA.

Example 1:

Convert the NFA with ε into its equivalent DFA.



Solution:
Let us obtain ε-closure of each state.

1. ε-closure {q0} = {q0, q1, q2}
2. ε-closure {q1} = {q1}
3. ε-closure {q2} = {q2}
4. ε-closure {q3} = {q3}
5. ε-closure {q4} = {q4}

Now, let ε-closure {q0} = {q0, q1, q2} be state A.
Hence

δ'(A, 0) = ε-closure {δ((q0, q1, q2), 0) }
= ε-closure {δ(q0, 0) ∪ δ(q1, 0) ∪ δ(q2, 0) }
= ε-closure {q3}
= {q3} call it as state B.

δ'(A, 1) = ε-closure {δ((q0, q1, q2), 1) }
= ε-closure {δ((q0, 1) ∪ δ(q1, 1) ∪ δ(q2, 1) }
= ε-closure {q3}
= {q3} = B.

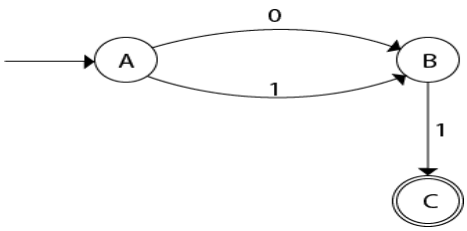
Now,

δ'(B, 0) = ε-closure {δ(q3, 0) }
= ϕ
δ'(B, 1) = ε-closure {δ(q3, 1) }
= ε-closure {q4}
= {q4} i.e. state C

For state C:

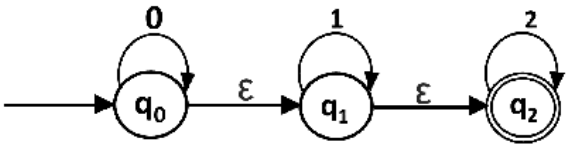
1. δ'(C, 0) = ε-closure {δ(q4, 0) } = ϕ
2. δ'(C, 1) = ε-closure {δ(q4, 1) } = ϕ

The DFA will be,



Example 2:

Convert the given NFA into its equivalent DFA.



Solution: Let us obtain the ε-closure of each state.

1. ε-closure(q0) = {q0, q1, q2}

2. ε-closure(q1) = {q1, q2}
3. ε-closure(q2) = {q2}

Now we will obtain δ' transition. Let ε-closure(q0) = {q0, q1, q2} call it as **state A**.

```
δ'(A, 0) = ε-closure{δ((q0, q1, q2), 0)}
         = ε-closure{δ(q0, 0) ∪ δ(q1, 0) ∪ δ(q2, 0)}
         = ε-closure{q0}
         = {q0, q1, q2}

δ'(A, 1) = ε-closure{δ((q0, q1, q2), 1)}
         = ε-closure{δ(q0, 1) ∪ δ(q1, 1) ∪ δ(q2, 1)}
         = ε-closure{q1}
         = {q1, q2}          call it as state B

δ'(A, 2) = ε-closure{δ((q0, q1, q2), 2)}
         = ε-closure{δ(q0, 2) ∪ δ(q1, 2) ∪ δ(q2, 2)}
         = ε-closure{q2}
         = {q2}          call it state C
```

Thus we have obtained

1. δ'(A, 0) = A
2. δ'(A, 1) = B
3. δ'(A, 2) = C

Now we will find the transitions on states B and C for each input.

Hence

δ'(B, 0) = ε-closure{δ((q1, q2), 0)}
= ε-closure{δ(q1, 0) ∪ δ(q2, 0)}
= ε-closure{ϕ}
= ϕ

δ'(B, 1) = ε-closure{δ((q1, q2), 1)}
= ε-closure{δ(q1, 1) ∪ δ(q2, 1)}
= ε-closure{q1}
= {q1, q2} **i.e. state B itself**

δ'(B, 2) = ε-closure{δ((q1, q2), 2)}
= ε-closure{δ(q1, 2) ∪ δ(q2, 2)}
= ε-closure{q2}
= {q2} **i.e. state C itself**

Thus we have obtained

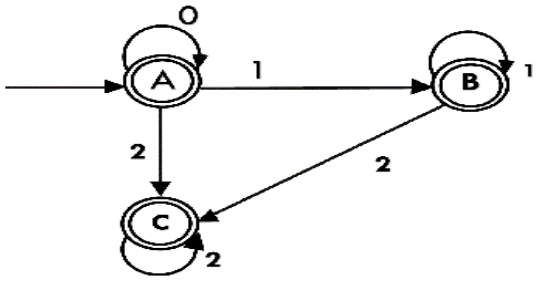
1. δ'(B, 0) = ϕ
2. δ'(B, 1) = B
3. δ'(B, 2) = C

Now we will obtain transitions for C:

δ'(C, 0) = ε-closure{δ(q2, 0)}
= ε-closure{ϕ}
= ϕ

δ'(C, 1) = ε-closure{δ(q2, 1)}
= ε-closure{ϕ}
= ϕ

δ'(C, 2) = ε-closure{δ(q2, 2)}
= {q2}



Hence the DFA is

As A = {q0, q1, q2} in which final state q2 lies hence A is final state. B = {q1, q2} in which the state q2 lies hence B is also final state. C = {q2}, the state q2 lies hence C is also a final state.

- **FA WITH OUTPUT**
  1. Moore Machine
  2. Mealy Machine

In case of Moore or Mealy machines, they do not require a final state. It has an initial state

i) Moore Machine

A Moore machine is a finite state machine that has an output value rather than a final state. For a given input, the machine generates a corresponding output. The output of the Moore machine depends only on the present state of the FA.

Output is represented with each input state separated by /.

Inputs are always outside the states and outputs are inside the states

Formal theorem

The Moore machine is a 6 tuple machine (Q, Σ, q0, Δ, δ, λ)

Q: This is a set of states.

Σ: This is a set of input alphabets.

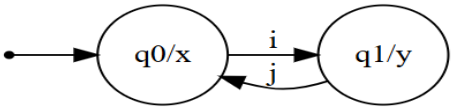
*q*0: This is the initial state.

Δ: This is a set of output states.

δ: This is a transition function, that is: Q×Σ→Q

λ: This is an output function, that is: Q→Δ

The Moore machine forms an FA of the type:

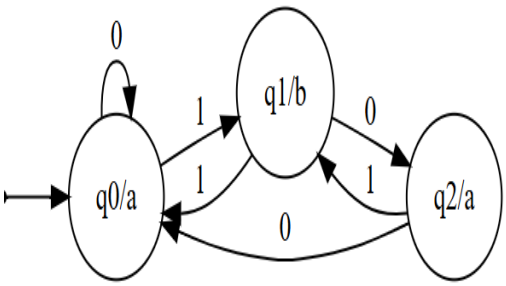


An example format of a Moore machine.

In the machine above, the transition on input **i** will give an output **y** from the state q0. Moreover, from q1 a transition on input **j** will provide an output **x**

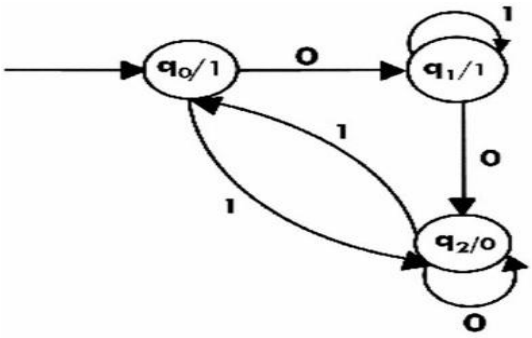
If the input string is of length **n**, the output produced by a Moore machine will be of length **n+1**

Example 1



current State	next State for 0	next State for 1	Output
q0	q0	q1	a
q1	q2	q0	b
q2	q0	q1	a

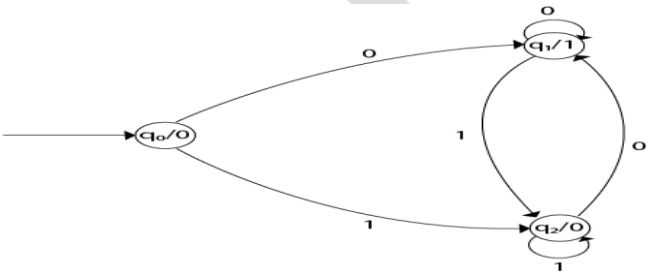
Example 2 :



Current State	Next State		Output
	0	**1**	
q0	q1	q2	1
q1	q2	q1	1
q2	q2	q0	0

Input − 010
Transition − δ (q0,0) => δ(q1,1) => δ(q1,0) => q2
Output – 1110 (1 for q0, 1 for q1, again 1 for q1, 0 for q2)

Example :
Design a Moore machine to generate 1's complement of a given binary number.



ii) Mealy machine

A **Mealy machine** is a finite state machine that has an output value rather than a final state. For a given input, the machine generates a corresponding output. The output of the Mealy machine depends on the present state of the FA as well as the current input symbol.

Mealy machines determine the output against the given input.

Formal theorem

The Mealy machine is a 6 tuple machine (Q,Σ,q0,Δ,δ,λ)

Q : This is a set of states.

Σ: This is a set of input alphabets.

*q*0: This is an initial state.

Δ: This is a set of output states.

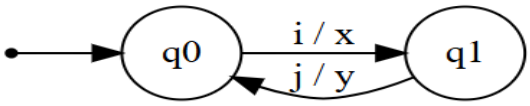
δ: This is a transition function, that is: Q×Σ→Q

λ: This is an output function, that is: $Q \times \Sigma \rightarrow \Delta$

The output function means that for every transition at a particular state, there is a corresponding output associated with it.

Formation of the machine

The Mealy machine forms an FA of the type:



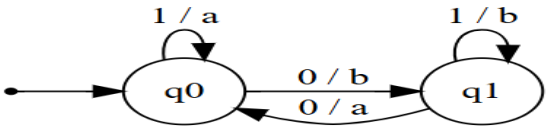
An example format of a Mealy machine

In the machine above, the transition on input i will give an output x from the state q0. Moreover, from q1 a transition on input j will provide an output y.

If the input string is of length n, the output produced by a Mealy machine will also be of length n.

The Mealy machine is faster than the Moore machine as it is asynchronous to the fluctuations of a clock pulse.

Example :



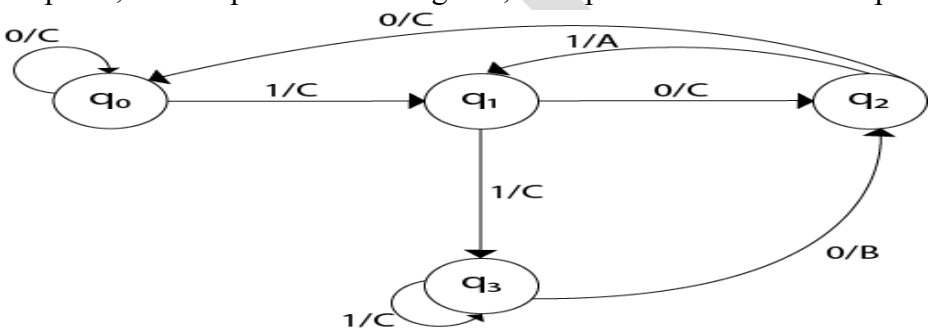
An example Mealy machine.

Now, we take an input string in $\Sigma\{0,1\}$ and see its output in $\{a,b\}$.

Current State	next State for 0	Output at 0	next State for 1	Output at 1
q0	q1	B	q0	a
q1	q0	A	q1	b

Example 1:

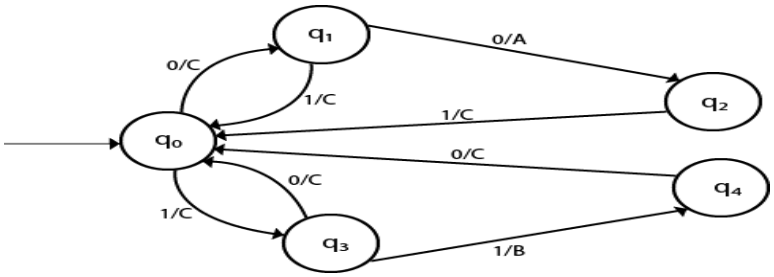
Design a Mealy machine for a binary input sequence such that if it has a substring 101, the machine output A, if the input has substring 110, it outputs B otherwise it outputs C.



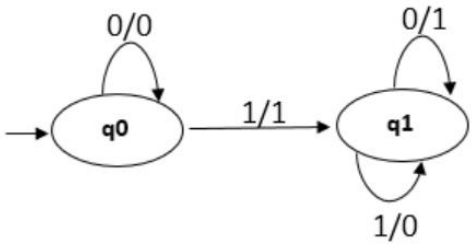
Example 2:

Design a mealy machine that scans sequence of input of 0 and 1 and generates output 'A' if the input string terminates in 00, output 'B' if the string terminates in 11, and output 'C' otherwise.

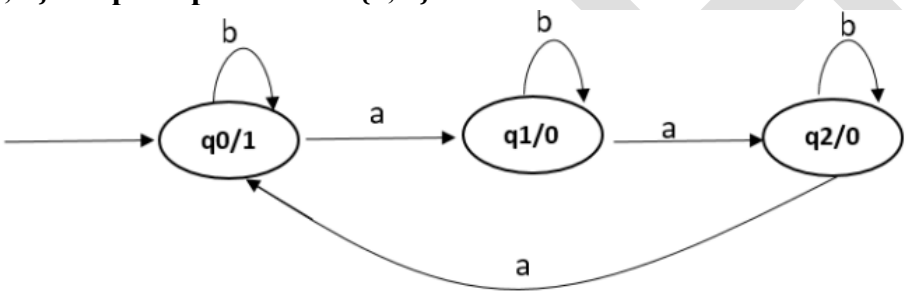
Solution: The mealy machine will be:



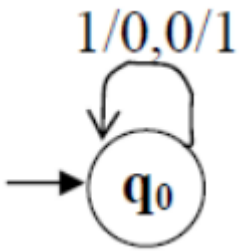
Construct a Mealy machine that take set of all strings over alphabet {0, 1} and make 2's complement of an input bit string in binary number. Sol: Input alphabet: Σ = {0, 1} Output alphabet: Γ = {0,1}



Construct a Moore machine that take set of all strings over alphabet {a, b} as input and produce '1' as output if number of a's in string mod 3=0 otherwise produce '0'. Sol: Input alphabet: Σ = {a, b} Output alphabet: Γ = {0, 1}



The following Mealy machine prints out the 1's complement of an input bit string.in other words it flips each digit from a 0 to a 1 or from a 1 to a 0.



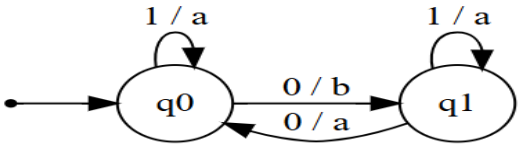
- **CONVERSION OF MEALY MACHINE TO MOORE MACHINE**

The steps involved in this method are:

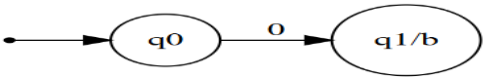
1. Select the initial state and draw the transitions by adding the output symbol of that transition to the destination state.
2. Repeat the process for every single transition.
3. If a conflict occurs, that is, if a state requires to give two different outputs, make a duplicate state.
4. Add transitions for all duplicate states on all input symbols.

Example：

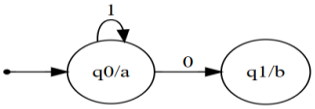
Let's take a simple Mealy machine:



1. We take the initial state and draw its first transition:

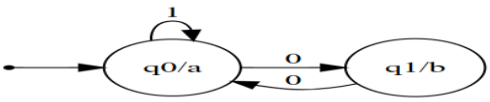


The transition had an output b , that is now transferred to our destination node $q1$. Now, for the second transition of state q0



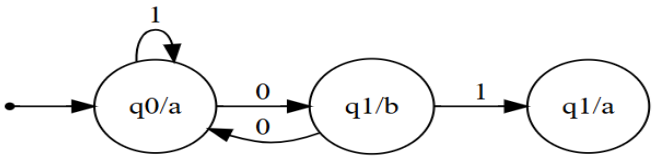
Step 1: Draw remaining transitions of the initial state

2. We repeat the process:



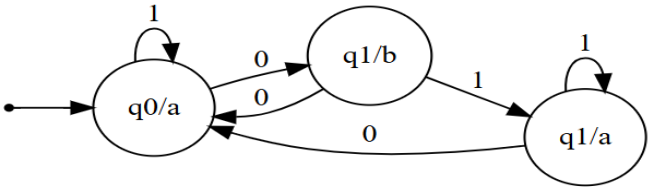
Step 2: Repeat the process

3. We encounter a conflict on the transition $q1 \rightarrow q1$ as it has an output a , but the state $q1$ already has an input b. So we make a duplicate state:



Step 3: Add duplicate states when conflict occurs

4. Now we add transitions to the new state:



Step 4: Add transitions to the new state

For example, we know from the Mealy machine that q1$q1$ at 00 goes to q0$q0$ and gives an output aa, so we use the same logic to form the new transition.

b) By transition table

The steps involved in this method are:

1. Find the states that have more than one output associated with them in each column. For such states, divide them into two.
2. Rewrite the transition table, including the new states, and add an output column. Then, fill in the destination state entries using the original table.
3. Check for the output of each state in the destination column and fill in the output column.
4. Remove the output symbols from the destination columns.

Example

Current State	Next State for 0	Output at 0	Next State for 1	Output at 1
q0	q1	b	q0	a
q1	q0	a	q1	a

1. We see that in the destination columns, the state q1 has two different outputs for the input symbols; hence we will make them into two states q1a(q1 with output *a*) and q1b(q1 with output *b*).
2. Now let's form the transition table with relevant outputs

Current State	Next State for 0	Next State for 1	Output
q0	q1b, b	q0, a	
q1a	q0, a	q1a, a	
q1b	q0, a	q1a, a	

For example, the transition of q0 on 0 is towards $q1$ in the Mealy machine, but in this case, we have split q1 in two states. As a result, we see the corresponding output of that transition. In this case, the output is b; therefore, the destination will be q1b

3. Now we fill in the output column:

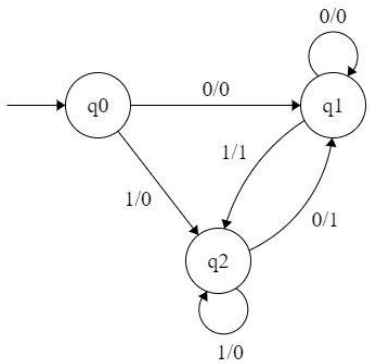
Current State	Next State for 0	Next State for 1	Output
q0	q1b, b	q0, a	a
q1a	q0, a	q1a, a	a
q1b	q0, a	q1a, a	b

For example, the q0 state has the output a in both columns of the destination; thus, the final output is also a.

4. Remove the output from the destination columns:

Current State	Next State for 0	Next State for 1	Output
q0	q1b	q0	a
q1a	q0	q1a	a
q1b	q0	q1a	b

Example 2



➢ CONVERSION OF MOORE MACHINE TO THE MEALY MACHINE

Moore machines can be converted to Mealy machines with a few simple steps. We know that Mealy machines are faster, so we may want to convert our Moore machine to a Mealy machine.

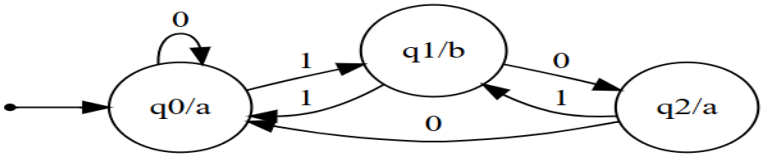
By transition diagram

The steps involved in this method are:

1. To redraw the original states without the output. The starting state remains the same.
2. Draw the transitions as they are on the original machine.
3. Add output to the transitions based on their destination's output symbol in the original machine.

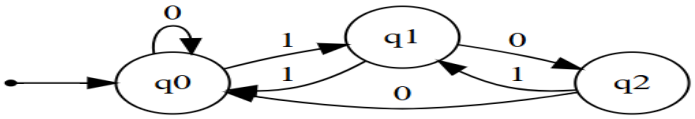
Example

Let's take a simple Moore machine:

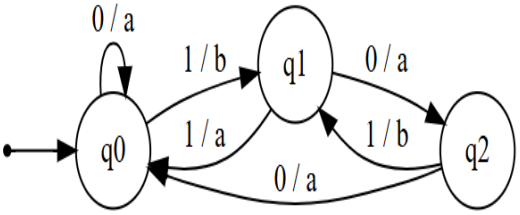


An example Moore machine

1. We redraw the states without the output:
2. Draw transitions:



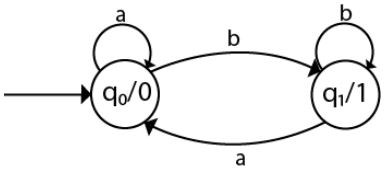
3. Now we add output to the transitions based on the transition's destination:



For example, the output of the transition q0→q1 is b, because the transition lands on q1 which had an output b in the original Moore machine.

The transformed machine above is our final Mealy machine.

Example



The transition table of given Moore machine is as follows:

Q	A	B	Output(λ)
q0	q0	q1	0
q1	q0	q1	1

The equivalent Mealy machine can be obtained as follows:

$\lambda'(q0, a) = \lambda(\delta(q0, a))$

$= \lambda(q0) \quad = 0$

$\lambda'(q0, b) = \lambda(\delta(q0, b))$

$= \lambda(q1) \quad = 1$

The λ for state q1 is as follows:

$\lambda'(q1, a) = \lambda(\delta(q1, a))$

$= \lambda(q0) \quad = 0$

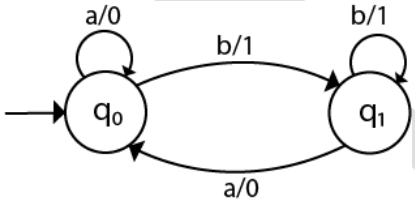
$\lambda'(q1, b) = \lambda(\delta(q1, b))$

$= \lambda(q1) \quad = 1$

Hence the transition table for the Mealy machine can be drawn as follows:

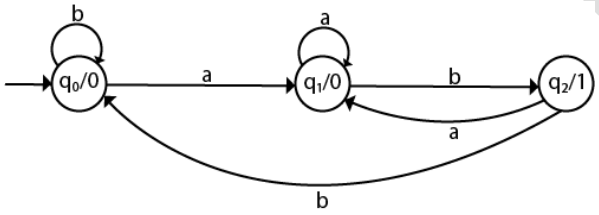
Σ / Q	Input 0		Input 1	
	State	O/P	State	O/P
q_0	q_0	0	q_1	1
q_1	q_0	0	q_1	1

The equivalent Mealy machine will be,

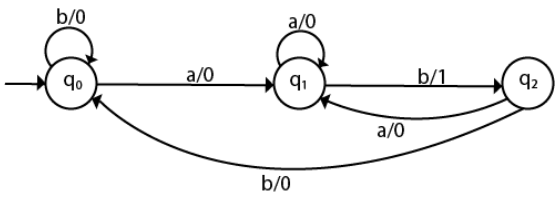


Example 2:

Convert the given Moore machine into its equivalent Mealy machine.



solution



- **Regular Expression**
  - The language accepted by finite automata can be easily described by simple algebraic expressions called Regular Expressions. It is the most effective way to represent any language.
  - Regular Expressions are an algebraic way to describe languages. The languages accepted by some regular expression are referred to as Regular languages.
  - A regular expression can also be described as a sequence of pattern that defines a string.
  - In short, a Regular Expression is written as RE.

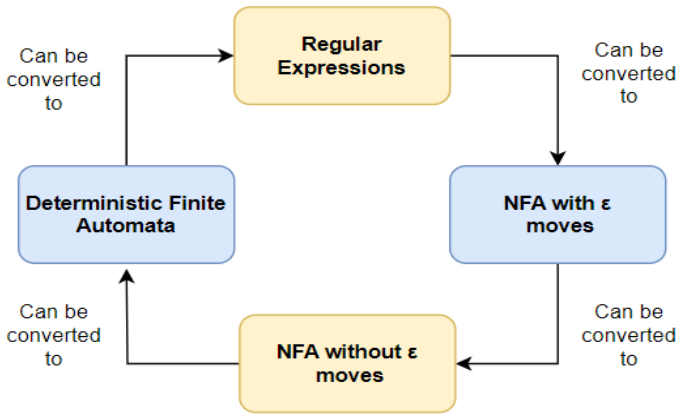
A **regular expression** represents a **regular language** if it follows these rules:

1. ϵ (epsilon) is a regular expression representing the language L = {ϵ}.
2. φ is a Regular Expression denoting an empty language. L = { }
3. Any symbol 'a' from the input alphabet Σ is a regular expression, representing the language {a}.
3. Union (a + b) is a regular expression if a and b are regular expressions, representing the language {a, b}.
4. Concatenation (ab) is a regular expression if a and b are regular expressions.
5. *Kleene star (a) is a regular expression**, meaning zero or more occurrences of 'a', forming a regular language. In a regular expression, a* means zero or more occurrences of a. It can generate {ε, a, aa, aaa, aaaa, aaaaa, ...}.
6. Positive closure(a) is regular expression a^+ means one or more occurrences of a. It can generate {a, aa, aaa, aaaa, aaaaa, ...}.
7. Parentheses: If E is a regular expression, then (E) is a regular expression, and L((E)) = L(E).

Parentheses may be used wherever needed to influence the grouping of operators.
We may remove parentheses by using precedence and associativity rules.

Relationship between Finite Automata and Regular Expression

Finite automata and regular expressions are closely related



➢ The Operators of Regular Expressions

Operators of Regular Expressions are the symbols used to construct and combine patterns in regular expressions (RE).

They define how strings are formed from symbols of an alphabet.

1. Union (OR) Operator (+ or |)

If R1 and R2 are regular expression, the R1+ R2 (or) R1 | R2 is regular expression denoting union of L(R1) and L(R2). That is, L(R1+ R2) = L(R1) U L(R2).

New language is set of strings that are in either L(R1) or L(R2) or both.

Ex: R1 = ab + c L(R1) = { ab, c}

R2 = a + b L(R2) = {a, b}

L(R1+ R2) = L(R1) U L(R2) = {ab, c, a, d}

2. Concatenation (AND) denoted with . (dot) or No symbold

If R1 and R2 are regular expression, the R1.R2 (or) R1R2 is regular expression denoting the concatenation of L(R1) and L(R2). That is, L(R1R2) = L(R1) L(R2).

Example : R1 = ab + c L(R1) = { ab, c}

R2 = a + b L(R2) = {a, b}

L(R1.R2) = L(R1).L(R2) = {aba, abb, ca, cb}

New language is set of strings that can be formed by taking any string in L(R1) and concatenating it with any string in L(R2)

3. Kleene Star (Closure) denoted with *

If R is regular expression then R* is a regular expression denoting closure of L(R). that is L(R*)=(L(R))*

Repeats zero or more times

Ex1: Let R = a L(R)={a} L(R*) = { ϵ, a, aa, aaa, ……}

Ex2 Let R = 0+1 L(R)={0,1} L(R*) = {ε, 0,1,00,11,01,10,000, … }

L* represents set of strings that can be formed by concatenating any number of strings from L , with possibly repetitions

- $L^* = \{\varepsilon\} \cup L^1 \cup L^2 \cup L^3 \cup \dots L^n$

4. Positive Closure +

If R is regular expression then R+ is a regular expression denoting positive closure of L(R). that is $L(R^+)=(L(R))^+$

Represents one or more repetitions

Similar to Kleene closure , but excludes empty string

Ex: Let R1 = a+ = L(R1) = { a, aa, aaa, ……}

$L^+ = L^1 \cup L^2 \cup L^3 \cup \dots L^n$

Parentheses may be used wherever needed to influence the grouping of operators.

Operator Precedence (Order of Evaluation)

1. *, +, ? (Highest)
2. Concatenation
3. | (Lowest)

- **Identity Rules of Regular Expressions**

Regular expression (RE) identity rules are algebraic laws, like those in math, that help simplify complex REs by defining equivalent forms

Some more identity rules are as given below:

I1: φ + R = R (Identity Law for union)

I2: φR = Rφ =φ (Annihilator (zero) rule for concatenation)

I3: εR = Rε = R (Identity Law for concatenation)

I4: ε* = ε and φ* = ε

I5: R + R = R (Idempotent Law)

I6: R*R* = R*

I7: RR* = R*R

I8: (R*)* = R*

I9: ε + RR* = R* = ε + R*R

I10: (PQ)*P = P(QP)*

I11: (P + Q)* = (P*Q*) = (P* + Q*)* (Demorgan law)

I12: (P +Q)R = PR + QR and R(P + Q) =RP + RQ (Distributed law)

- **Finite Automata and Regular Expression**

Finite automata are formal (or abstract) machines for *recognizing* patterns. *Regular expressions* are a formal notation for *generating* patterns.

A finite automaton (FA) is a computational model that recognizes patterns in input strings by transitioning through different states based on the input symbols

A regular expression is a concise notation used to describe the same patterns, essentially acting as a "formula" for the language a finite automaton can recognize;

Finite automaton is a machine that checks if a string matches a pattern

Regular expression is a way to write down that pattern using specific symbols and operators.

- **Arden's Theorem**

Arden's Theorem is a fundamental result in formal language theory and automata theory, particularly in the context of regular languages and finite Automata. It provides a method for finding the solutions to certain types of linear equations over regular languages. The theorem is named after Richard Arden, who introduced it in the context of solving equations involving regular expressions.

The following are the main applications of Arden's Theorem:

- It aids in determining the regular expression of finite Automata.
- Arden's theorem helps determine whether two regular expressions are equivalent.

Let P and Q be two regular expressions over an alphabet Σ. If P does not contain ϵ (the empty string), then the equation:

we get any equation in the form of R = Q + RP, then we can directly replace it with R = QP*

Proof of Arden's Theorem

1. Proof R = QP* is the unique solution of R = Q + RP

Let me take this equation again:

R = Q + RP

Now, replace R by R = Q + RP,

$R = Q + (Q + RP)P$

$= Q + QP + RP^2$

Again, replace R by R = Q + RP :-

$R = Q + QP + (Q + RP)\ P^2$

$= Q + QP + QP^2 + RP^3$

...

....

$= Q + QP + QP^2 + .. + QP^n + RP^{(n+1)}$

Now, replace R by R = QP*, we get,

$R = Q + QP + QP^2 + .. + QP^n + Q\ P^*\ P^{(n+1)}$

Taking Q as common,

$R = Q(\in + P + P^2 + .. + P^n + P^*P^{(n+1)})$

$= QP^*$ *[As* $\in + P + P^2 + .. + P^n + P^*P^{(n+1)}$ *represent the closure of P]*

Hence proved. Thus, R = QP* is the unique solution of the equation R = Q + RP.

2. proof R = QP* is the solution of R = Q + RP

$R = Q + RP$ *……(i)*

Now, replacing R by R = QP*, we get,

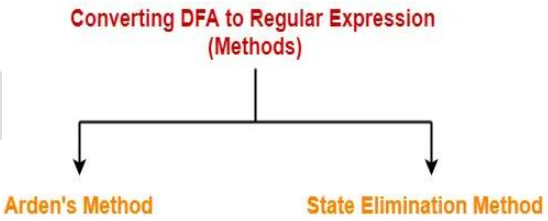
$R = Q + QP^*P$

Taking Q as common,

$R = Q(\in + P^*P) = QP^*$

(As we know that ∈ + R*R = R*). Hence proved. *Thus, R = QP* is the solution of the equation R = Q + RP.* Now, we have to prove that this is the only solution to this equation.

➢ CONVERTING FA TO RE



1. Using Arden's Theorem

Let P and Q be the two regular expressions.

If P does not contain null string, then the following equation in R, `R = Q + RP`,

Which has a unique solution by `R = QP*`

To use Arden's Theorem, following conditions must be satisfied-

- The transition diagram must not have any ∈ transitions.
- There must be only a single initial state.

Step-01:

Form a equation for each state considering the transitions which comes towards that state. i.e. create the equations in the given form for all the states.

Add '∈' in the equation of initial state.

Let α_{ji} represents the transition from q_j to q_i .

Calculate qi such that $q_i = q_j\ \alpha_{ji}$

If q_j is a start state then we have: $q_i = q_j\ \alpha_{ji} + \varepsilon$

Step-02:

Bring final state in the form R = Q + RP to get the required regular expression.

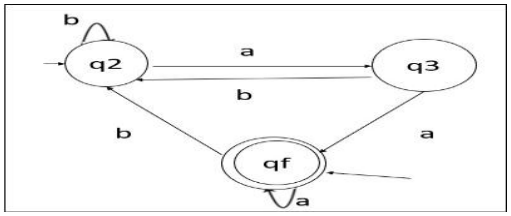
If there exists multiple final states, then-

Write a regular expression for each final state separately.

Add (Union) all the regular expressions to get the final regular expression.

Arden's Theorem can be used to find a regular expression for both DFA and NFA. Arden's Method is not capable of converting Ɛ-NFA.

Example



q 1= q1a + q3a + € (€ move is because q1is the initial state)

q2 = q1b + q2b + q3b

q3 = q2a

Now, we will solve these equations, as shown below−

→ q2 = q1b + q2b + q3b

= q1b + q2b + (q2a)b (Substituting value of q3)

= q1b + q2(b + ab)

= q1b (b + ab) * (Applying Arden's Theorem)

→q1 = q1a + q3a + €

= q1a + q2aa + € (Substituting value of q3)

= q1a + q1b(b + ab*)aa + € (Substituting value of q2)

= q1(a + b(b + ab) *aa) + €

= € (a+ b(b + ab) *aa) *

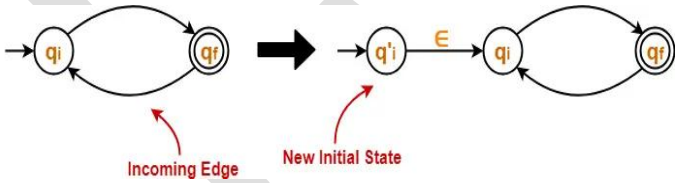
= (a + b(b + ab) *aa) *

Hence, the regular expression is (a + b(b + ab) *aa) *.

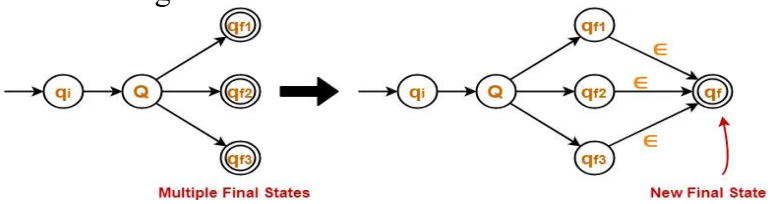
2. State Elimination Method

This method involves the following steps in finding the regular expression for any given DFA-

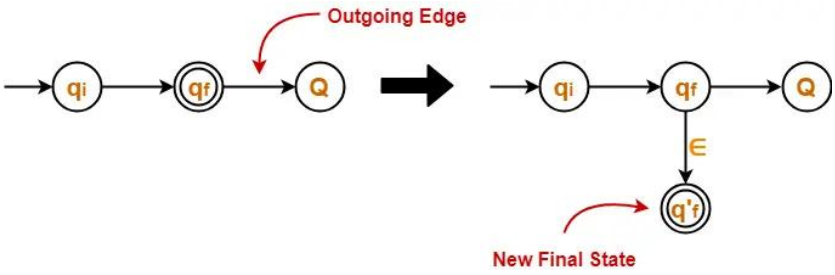
- Step-01:
  The initial state of the DFA must not have any incoming edge.
  If there exists any incoming edge to the initial state, then create a new initial state having no incoming edge to it.



- Step-02:
  There must exist only one final state in the DFA.
  If there exist multiple final states in the DFA, then convert all the final states into non-final states and create a new single final state.



- Step-03:
  The final state of the DFA must not have any outgoing edge.
  If there exists any outgoing edge from the final state, then create a new final state having no outgoing edge from it.



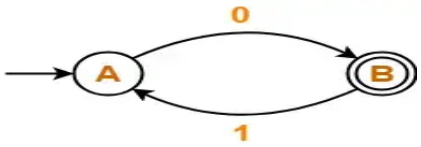
- Step-04:
  Eliminate all the intermediate states one by one.
  These states may be eliminated in any order.

In the end,

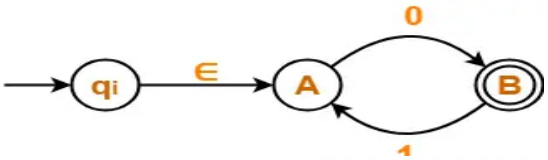
- Only an initial state going to the final state will be left.
- The cost of this transition is the required regular expression.

The state elimination method can be applied to any finite automata (NFA, ∈-NFA, DFA etc)

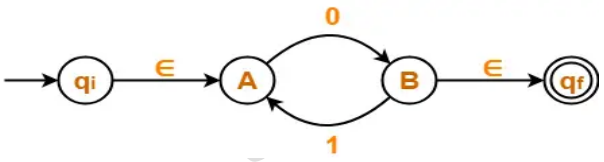
Example for FA to RE conversion



- Initial state A has an incoming edge.
- So, we create a new initial state q_i.



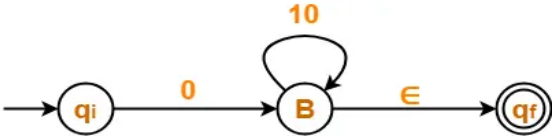
- Final state B has an outgoing edge.
- So, we create a new final state q_f



The resulting DFA is-
Now, we start eliminating the intermediate states.
First, let us eliminate state A.

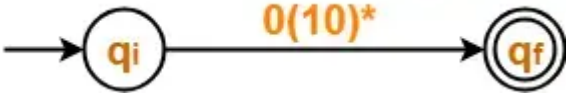
- There is a path going from state q_i to state B via state A.
- So, after eliminating state A, we put a direct path from state q_i to state B having cost ∈.0 = 0
- There is a loop on state B using state A.
- So, after eliminating state A, we put a direct loop on state B having cost 1.0 = 10.

Eliminating state A, we get-



Now, let us eliminate state B.

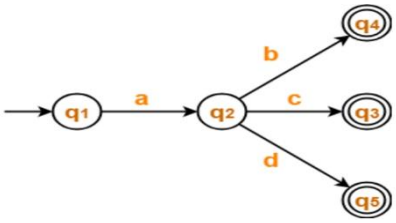
- There is a path going from state q_i to state q_f via state B.
- So, after eliminating state B, we put a direct path from state q_i to state q_f having cost 0.(10)*.∈ = 0(10)*



Eliminating state B, we get-
Regular Expression = 0(10)*
In the above question,

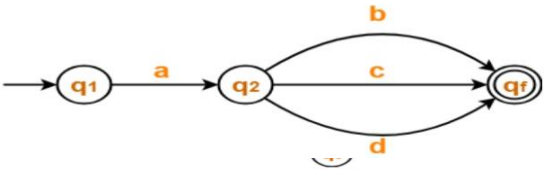
- If we first eliminate state B and then state A, then regular expression would be = (01)*0.
- This is also the same and correct.

Example 2 : For the following DFA, find its equivalent regular expression?

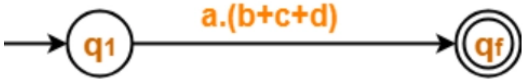


Step 1: We will first multiple final states into a single final state
The resulting DFA after step 1 will be

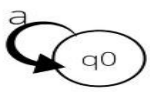
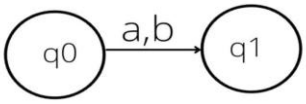
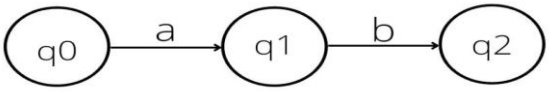
Step 2: Now, eliminate all the intermediary steps.
After eliminating state q4, q3, q5, DFA will look like



Further, after eliminating state q2, DFA will look like,



Finally, we are left with only the initial & final state, and here cost of this transition is the required regular expression.
So, *Regular expression = a.(b+c+d)*
Conversion of RE to FA
Some standard rules help in the conversion of RE to NFA are:-

Kleene's Closure (*) can be eliminated by introducing self-loops on states. If R = **a*,** it can be represented as:	
Union operator (+) can be eliminated by introducing parallel edges between the two states If R= **a+b,** it can be represented as	
The concatenation operator ('.' or no operator at all) can be eliminated by introducing a new state between the states.	

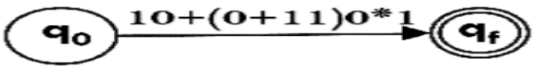
a) Conversion of RE to FA using Subset Method

To convert the RE to FA, we are going to use a method called the subset method. This method is used to obtain FA from the given regular expression. This method is given below:
Step 1: Design a transition diagram for given regular expression, using NFA with ε moves.
Step 2: Convert this NFA with ε to NFA without ε. (ε-NFA to NFA)
Step 3: Convert the obtained NFA to equivalent DFA. (NFA to DFA)

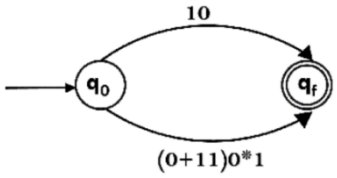
Example 1: Design a FA from given regular expression `10 + (0 + 11)0* 1.`

Solution: First we will construct the transition diagram for a give

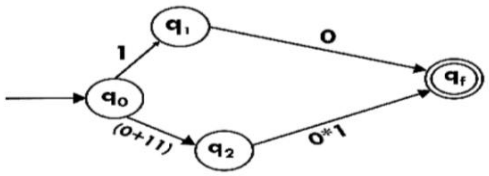
Step 1:



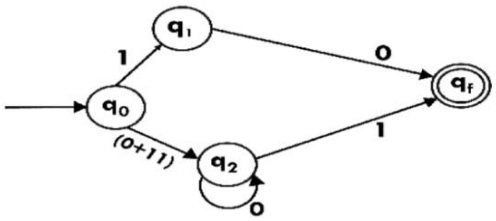
Step 2:



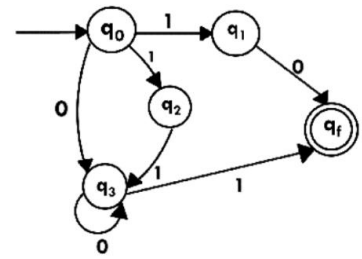
Step 3:



Step 4:



Step 5:



Now we have got NFA without ε. Now we will convert it into required DFA for that, we will first write a transition table for this NFA

State	0	1
→q0	q3	{q1, q2}
q1	qf	ϕ
q2	ϕ	q3
q3	q3	qf
*qf	ϕ	ϕ

,

The equivalent DFA will be:

State	0	1
→[q0]	[q3]	[q1, q2]
[q1]	[qf]	ϕ
[q2]	ϕ	[q3]
[q3]	[q3]	[qf]
[q1, q2]	[qf]	[qf]
*[qf]	ϕ	ϕ

b) State Decomposition Method

Theorem: Every language defined by a regular expression is also defined by a Finite Automata.

Proof: Let's assume L = L(R) for a regular expression R. We prove that L = L(M) for some ε-NFA with:

1) Exactly one accepting state.

2) No incoming edges at the initial state.

3) No outgoing edges at the accepting state.

The proof is done by structural induction on R by following the steps below:

Step 1:

Create a starting state, say q_1, and a final state, say q_2. Label the transition q_1 to q_2 as the given regular expression, R, as in Fig 1. But, if R is $(Q)^*$, Kleene's closure of another regular expression Q, then create a single initial state, which will also be the final state, as in Fig 2.

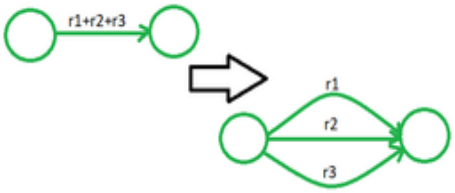




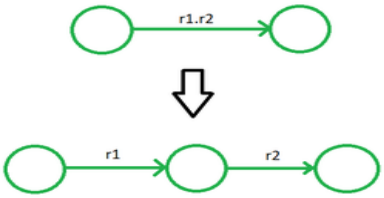
Step 2:

Repeat the following rules (state decomposition method) by considering the least precedency regular expression operator first until no operator is left in the expression. Precedence of operators in regular expressions is defined as Union < Concatenation < Kleene's Closure.

Union operator (+) can be eliminated by introducing parallel edges between the two states as follows.

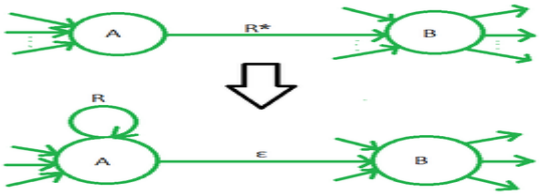


The concatenation operator ('.' or *no operator at all*) can be eliminated by introducing a new state between the states as follows.

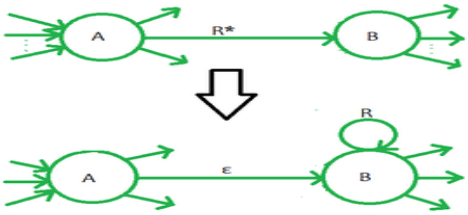


Kleene's Closure (*) can be eliminated by introducing self-loops on states based on the following conditions:

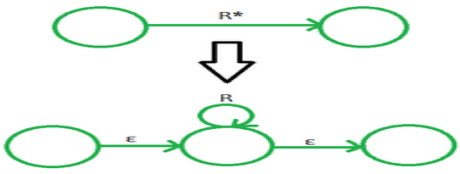
1. If there is only one outgoing edge at the left-most state, i.e., A in transition A -> B, then introduce self-loop on state A and label edge A to B as an ε-transition, as shown in Fig .



2.Else if there is only one incoming edge at the right-most state, i.e., B in transition A -> B, then introduce self-loop on state B and label edge A to B as an ε-transition, as shown in Fig



3. Else introduce a new state between two states having self-loop labeled as the expression. The new state will have ε-transitions with the previous states as follows, as shown in Fig.

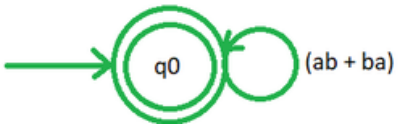


Example:

Construct Finite Automata for the regular expression, $R = (ab + ba)^*$

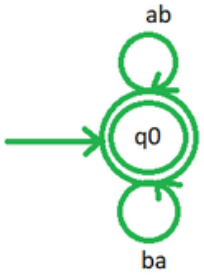
Solution:

Step 1: As the given expression, R, is of the form $(Q)^*$, so we will create a single initial state that will also be the final state, having self-loop labelled (ab + ba)

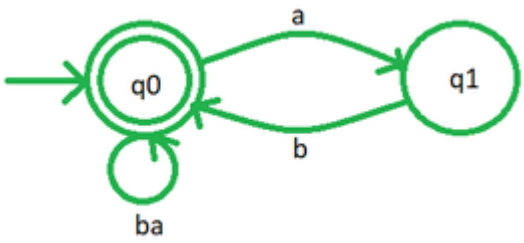


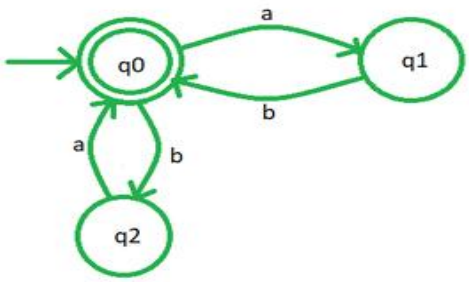
Step2:

A. As the least precedency operator in the expression is a union(+). So we will introduce parallel edges (parallel self-loops here) for 'ab' and 'ba',



B. Now we have two labels with concatenation operators (no operator mentioned between two variables is concatenation), so we remove them one by one by introducing new states, q_1 and q_2 as shown in Fig

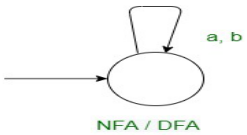




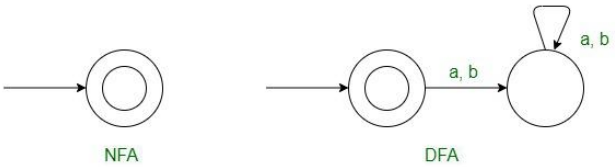
Step 3: As no operators are left, we can say that Fig is the required finite automata (NFA).

- **Basic FA (DFA, NFA) for regular expressions Σ = {a,b}**

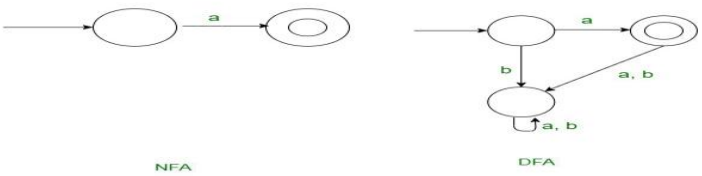
Case-1: When r = Φ, then FA will be as follows.



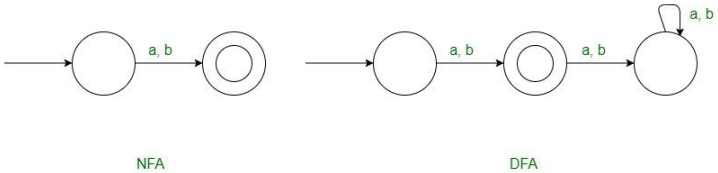
Case-2: When **r = ε**, then FA will be as follows.



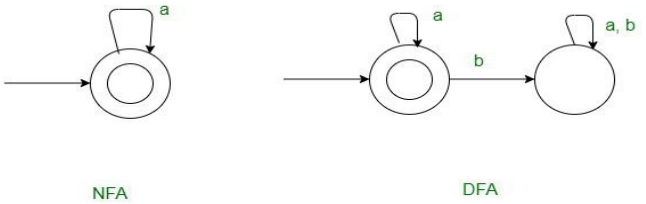
Case-3: When **r = a**, then FA will be as follows.



Case-4: When **r = a+b** , then FA will be as follows.

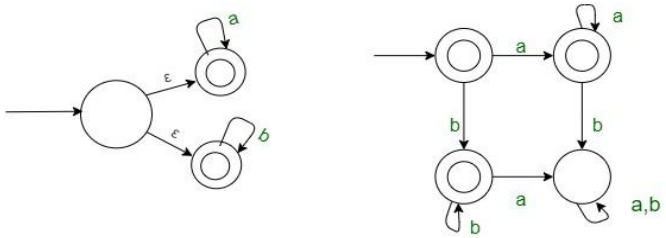


Case-5: When **r = r = a*** , then FA will be as follows.



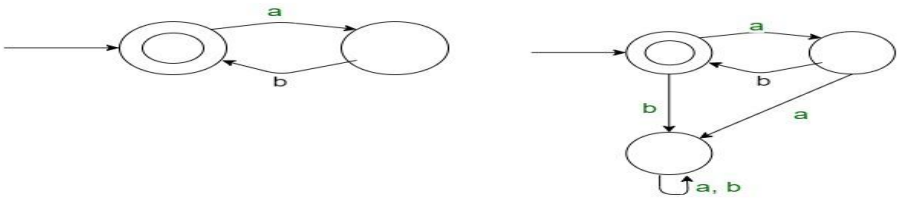
NFA DFA

Case-6: When **r = a* + b***, then FA will be as follows.



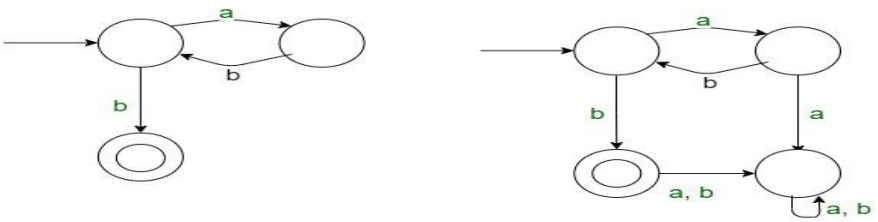
NFA DFA

Case-7: When **r = (ab)***, then FA will be as follows.



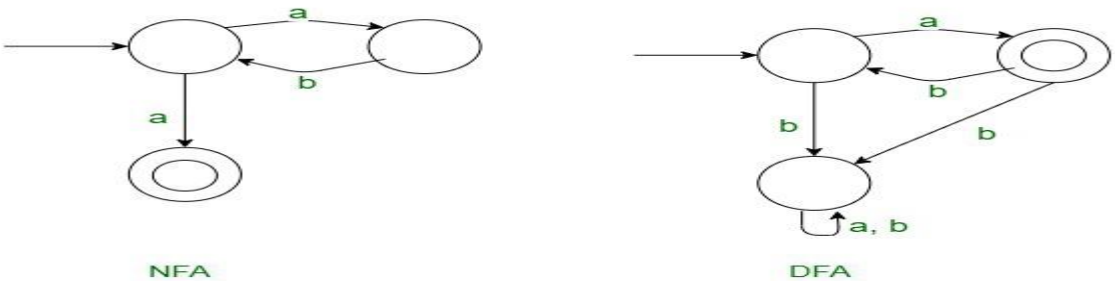
NFA DFA

Case-8 : When **r = (ab)*b**, then FA will be as follows.



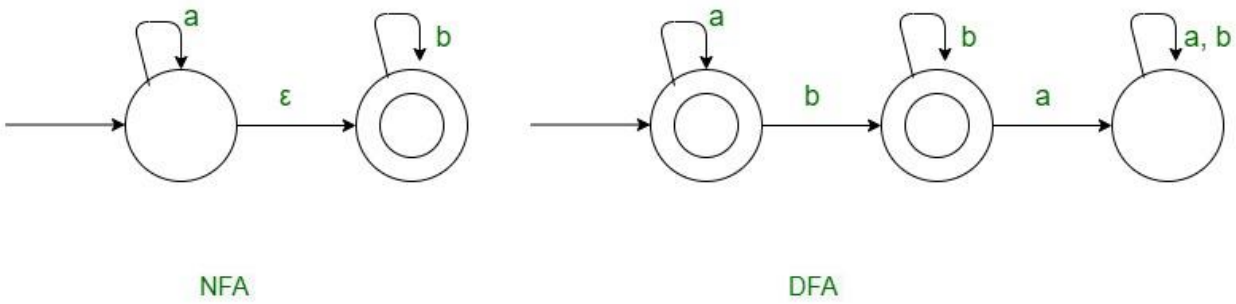
NFA DFA

Case-9: When **r = (ab)*a**, then FA will be as follows.



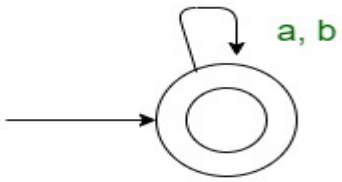
NFA DFA

Case-10 : When **r = a*b***, then FA will be as follows.



NFA DFA

Case-11: When **r = (a+b)***, then FA will be as follows.



NFA / DFA

➢ **Applications of Regular Expression**

Regular expressions (regex) are powerful tools used for pattern matching and manipulation of text. They are widely used in various applications, including:

1. **Text Search and Validation**: Regular expressions are commonly used to search for specific patterns or validate input. For example, you can use regex to search for email addresses, phone numbers, or URLs in a text document.
2. **Data Extraction**: Regular expressions can be used to extract specific data from a larger text. For instance, you can extract all the dates or numbers from a document using regex patterns.
3. **Data Cleaning**: Regular expressions are useful for cleaning and transforming data. You can remove unwanted characters, replace specific patterns, or reformat data using regex. This is particularly helpful when dealing with messy or unstructured data.
4. **String Manipulation**: Regular expressions enable you to manipulate strings by replacing or rearranging specific patterns. For example, you can use regex to find and replace words or phrases in a document.
5. **Form Validation**: Regular expressions are commonly used in web development to validate form inputs. You can use regex patterns to ensure that user inputs match specific criteria, such as a valid email address or a strong password.
6. **Programming and Scripting**: Regular expressions are supported in many programming languages and scripting tools. They provide a concise and powerful way to perform complex string operations, such as searching, replacing, and splitting strings.
7. **Lexical Analysis**: Regular expressions are fundamental in lexical analysis, which is the process of tokenizing and parsing source code or other structured text. They are used to define the grammar rules and patterns for identifying different tokens in a programming language.

- **Algebraic Laws for Regular Expressions**

Let R, S, T are regular expressions

Commutative Law

It is the property of an operator that allows us to switch the order of its operands and get the same result. For an RE, the commutative law for union is $R + S = S + R$

But, the commutative law for concatenation does not hold good, i.e., $RS \neq SR$

Associativity Law

It is the property of an operator that allows us to regroup the operands when operator is applied twice.

Similarly, for an RE, the associative law for union is $(R+S)+T = R+(S+T)$

The associative law for concatenation states that $(RS)T=R(ST)$

Distributive Law

It involves two operators and asserts that one operator can be pushed down to be applied to each argument of the operator individually.

In the case of RE, the distributive law is stated in two forms, that are,

Left Distributive law of concatenation over union which states that $R(S+T) =RS+RT$

Right Distributive law of concatenation over union which states that

$(S+T)R = SR+ TR$

Idempotent Law

An operator is said to be idempotent if the result of applying it to two of the same values as arguments is that value itself.

For an RE, the idempotent law for union is $R + R = R$ which means if we take two the union of two identical expressions, we can replace them by one copy of the expression.

Identity Law

An identity for an operator is a value that when the operator is applied to the identity and some other value, the result is the other value. For regular expressions we have:

$\emptyset + R = R + \emptyset = R$ This law asserts that $\emptyset$ is the identity for union

$\varepsilon.R = R. \varepsilon = R$ This law asserts that ε is the identity for concatenation

Annihilator

An annihilator for an operator is value that when the operator is applied to the annihilator and some other value, the result is the annihilator.

$\emptyset.R = R. \emptyset = \emptyset$ This law asserts that $\emptyset$ is the annihilator for concatenation

DeMorgan's Law

$(R + S)^* = (R^*S^*)^*$

Closures Laws

- $(R^*)^*=R^*$: This law states that closing an expression that is already closed doesn't change the language
- $\emptyset^* = \varepsilon$, The closure of $\emptyset$ contains only the string ε
- $\varepsilon^* = \varepsilon$ The only string that can be formed by concatenating any number of copies of the empty string is the empty string itself
- $R^* = \varepsilon + RR^*$

Absorption Laws

$R + R^* = R^*$

$R^* + RR^* = R^*$

➢ PUMPING LEMMA FOR REGULAR LANGUAGES

There are two Pumping Lemmas, which are defined for

1. Regular Languages, and
2. Context – Free Languages

Theorem

Let L be a regular language. Then there exists a constant 'c' such that for every string w in L $|w| \geq c$

We can break w into three strings, w = xyz, such that

- $|y| > 0$
- $|xy| \leq c$

• For all $k \geq 0$, the string xy^kz is also in L.

In simple terms, this means that if a string y is 'pumped', i.e., if y is inserted any number of times, the resultant string still remains in L.

Pumping Lemma is used as a proof for *irregularity* of a language.

Thus, if a language is regular, it always satisfies pumping lemma.

If there exists at least one string made from pumping which is not in L, then L is surely not regular.

It is important to note that pumping lemma is not used for proving whether a language is regular. It is rather used for proving if the language is not regular.

The word pumping refers to generating many input strings by pushing a symbol in an input string repeatedly.

Applications of Pumping Lemma

- If L is regular, it satisfies Pumping Lemma.
- If L does not satisfy Pumping Lemma, it is non-regular. If at least one string is made from pumping, not in language A, then A is not regular.

We use the CONTRADICTION method to prove that a language is not Regular.

Steps to prove that a language is not Regular using Pumping Lemma:

Step 1: Assume that Language A is Regular.
Step 2: It has to have a Pumping Length (say P).
Step 3: All strings longer than P can be pumped $|S| \geq P$.
Step 4: Now, find a string 'S' in A such that $|S| \geq P$.
Step 5: Divide S into x y z strings.
Step 6: Show that $xy^iz \notin A$ for some i.
Step 7: Then consider how S can be divided into x y z.
Step 8: Show that none of the above strings satisfies all three pumping conditions simultaneously.
Step 9: S cannot be pumped == CONTRADICTION.

Let's apply these above steps to check whether a Language is not Regular with the help of Pumping Lemma.

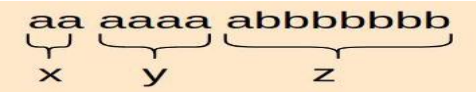
For example, let us prove L01 = $\{0^n1^n \mid n \geq 0\}$ is irregular.

Solution: We will follow the steps we have learned above to prove this.

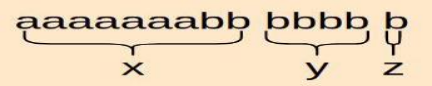
Assume that A is Regular and has a Pumping length = P.

Let a string **$S = a^pb^p$.**

Now divide the S into the parts, x y z.
To divide the S, let's take the value of P = 7.
Therefore, S = aaaaaaabbbbbbb (by putting P=7 in S = a^pb^p).
Case 1: Y consists of a string having the letter only 'a'.



Case 2: Y consists of a string having the letter only 'b'.



Case 3: Y consists of a string with the letters 'a' and 'b'.



For all the above cases, we need to show $xy^iz \notin A$ for some i.
Let the value of i = 2. **$xy^iz \Rightarrow xy^2z$**
In Case 1. **xy^2z = aa aaaa aaaa abbbbbbb**
No of 'a' = 11, No. of 'b' = 7.
Since the **No of 'a' != No. of 'b'**, but the original language has an equal number of 'a' and 'b'; therefore, this string will not lie in our language.
In Case 2. **xy^2z = aaaaaaabb bbbb bbbb b**
No of 'a' = 7, No. of 'b' = 11.
Since the **No of 'a' != No. of 'b'**, but the original language has an equal number of 'a' and 'b'; therefore, this string will not lie in our language.
In Case 3. **xy^2z = aaaa aabb aabb bbbbb**
No of 'a' = 8, No. of 'b' = 9.
Since the **No of 'a' != No. of 'b'**, but the original language has an equal number of 'a' and 'b', and also, this string did not follow the a^nb^n pattern; therefore, this string will not lie in our language.
We can see at i = 2 all the above three strings do not lie in the language $A = \{a^nb^n \mid n \geq 0\}$.
Therefore, the language $A = \{a^nb^n \mid n \geq 0\}$ is not Regular.

- **PROPERTIES OF A REGULAR LANGUAGE**

  Language classes have two important kinds of properties:

  1. Decision properties. 2. Closure properties.

Closure properties

If a set of regular languages are combined using an operator, then the resulting language is also regular. This is different from Kleene closure

If we have two regular languages, L1 and L2, and L is generated by performing specific operations on L1 and L2, then L is also regular.

Closure properties used in Regular languages are as follows:

1. Union
2. Concatenation
3. Intersection
4. Reversal
5. Difference
6. Complementation
7. Homomorphism
8. Inverse Homomorphism

1.Union

Let's assume there are two regular languages, L1 and L2. The union of these two regular languages, L1∪L2, is also regular. Combining two languages, L1 and L2, by taking all strings that are in either L1 or L2. We can represent union as L1+L2.

Example

R1 = a(a+b)* → L1 = {a, abb, abba, aaa, ….} Strings starting with 'a'

R2 = (a+b)*b → L2 = { b, ab, aab, baaab, … } strings ending with 'b'

R3 = R1 + R2 R3 = a(a+b)* + (a+b)* | b Strings that start with a OR end with b

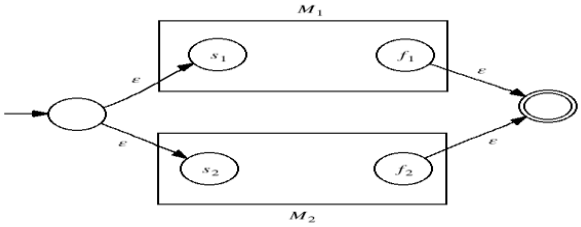
So: L3 = L1 ∪ L2 = { a, abb, abba, aaa , b, ab, aab, baaab, … }

- *The process for union*

Let's assume there are two languages, L1 and L2. We can perform the union of these two languages through the following steps:

1. Make the finite automaton for both L1 and L2 separately.
2. Add a new start state. Join the new start state to the automata's original start states by null transitions.

The final states remain the same as for the two automata.



2. Intersection

If there are two languages, L1 and L2, the intersection of these regular languages, L1∩L2 , is also regular. Creating a new language by taking all strings that are present in both L1 and L2.

$$L \cap M = \overline{\overline{L} \cup \overline{M}}$$

Example:

R1 = a(a+b)* → L1 = {a, abb, abba, aaa, ….} Strings starting with 'a'

R2 = (a+b)*b → L2 = { b, ab, aab, baaab, … } strings ending with 'b'

Intersection: L3 = L1 ∩ L2 Strings that start with a AND end with b

So, L3 = { ab, aaab, aabab, … } (ε is excluded because it doesn't end with a)

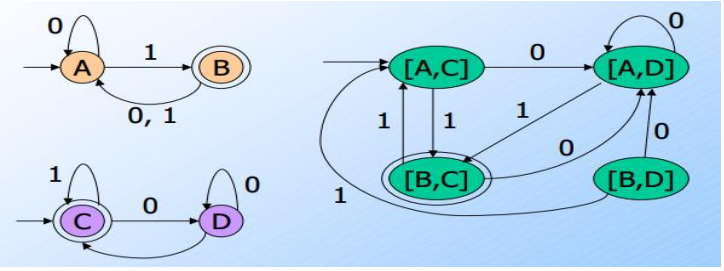
R3 = a(a+b)*b

This L3 is regular, because regular languages are closed under intersection

- *The process for the intersection*

The process for the intersection is slightly different from other properties. The steps to implement intersection are as follows:

1. Make the automatons for both languages.
2. Take the cross-product of all states from both the automatons.
3. The final state is the one that has the final state of the first automaton and the second automaton.



3. Concatenation

For concatenation, let's assume there are two regular languages, L1 and L2. The concatenation of these two languages, L1.L2 , is also regular. Combining two languages, L1 and L2, by creating all strings where a string from L1 is followed by a string from L2.

Example:

R1 = a(a+b)* → L1 = {a, abb, abba, aaa, ….} Strings starting with 'a'

R2 = (a+b)*b → L2 = { b, ab, aab, baaab, … } strings ending with 'b'

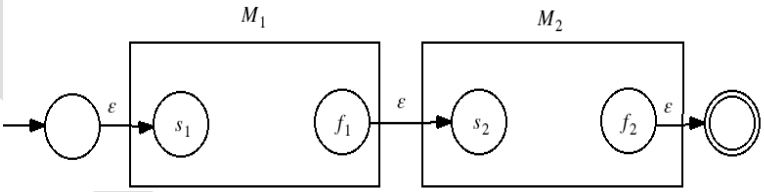
L1.L2 is All strings start with a and end with b

- *The process for concatenation*

Assume that there are two languages, L1 and L2. We can perform the concatenation of these two languages through the following steps:

1. Make the finite automaton for both L1 and L2 separately.
2. Suppose the order of concatenation is L1.L2. Make a null transition from the final state of L1 to the starting state of L2. Make the final state of L1 nonfinal.

The final state of L2 remains.



4. Kleene- *

If L is a language, the language L^* is the language of all strings that can be written as the concatenation of 0 or more strings taken from L.

If L is a regular language, L^* is also regular.

Example:

If R= (a), its language will be L= {a}

If R* = (a)*, then its language will be L* = { ε, a, aa, aaa, aaaa….}

Positive Closure

R is a regular expression whose language is L.R$^+$is a regular expression whose language is L^+.

Ex : If R= (a), its language will be L= {a}

If $R^+ = (a)^+$, then its language will be L^+ = {a, aa, aaa, aaaa….}

5. Complement

Creating a new language by taking all strings over the alphabet that are not present in the original language.

L1′ represents the complement of the regular language L1. Let's assume an alphabet, **Σ**, where **Σ*** contains the language, L1 . The complement is defined as **Σ*** − L1 , which is also regular.

Example:

Let Σ = {a, b} Let R = a(a+b)* i.e starting with 'a'

So L= { a, ab, aba, abbb, aa… }

Complement of L = Σ* – L all strings over {a, b}except strings starting with 'a'

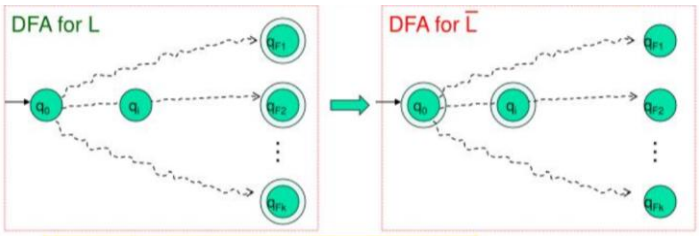
So: L' = { ε ,b, baaa, bbba, baab, bab, bb, … }

L' is also a regular language. R' = b(a+b)*+ε

- *The process for the complement*

There is one step for the complement process:

1. Make the accepting states non-accepting and vice versa.



6. Reversal

Let's assume a regular language, L1 . The reverse of L1 is represented as L^R. Creating a new language by reversing each string in the original language.

- *The process for reversal*

To reverse a language, we perform the following the steps:

1. Make a definite finite automaton for the regular language.
2. Make the final state the new initial state and the initial state the new final state.
3. Reverse the direction of the transitions.

If there are dangling states, we can remove them from the automaton.

Example: L = {0, 01, 100}; L^R = {0, 10, 001}

Example :

Let Σ = {a, b} Let R = a(a+b)* i.e starting with 'a'

So L= { a, ab, aba, abbb, aa… }

L^R = (a+b)*a strings that **end with a** L^R={a, ba, bbbba, ….. }

7. Difference

Consider the two languages, L1 and L2. The difference between these regular languages is shown by L1−L2. Creating a new language by taking all strings that are in L1 but not in L2.

This difference is L1∩ $\overline{L2}$ which means the intersection of L1 and the complement of L2.

Example :

Let R1 = a(a+b)* i.e starting with 'a' L1 = { a, ab, aba, abbb, aa… }

R2 = (a+b)*b i.e ending with 'b' L2 = { b, aaab, abbab,… }

L1- L2 =Strings that start with a BUT do NOT end with b **i.e.** Strings that start with a and end with a (or are just a).

R3 = a(a+b)* a + a

L3 = {a, abba, aaa, ababa, ….}

Note : L1- L2 ≠ L2 – L1

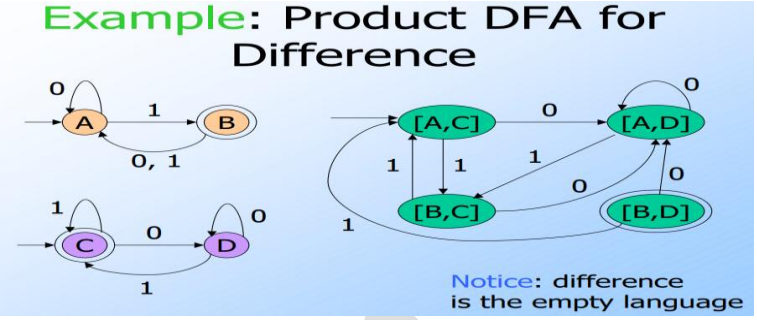
The process for difference

The process to take the difference between the two languages is as follows:

1. Make the automatons for the regular languages, L1 and L2.

2. Take the complement of the second language (the language that is to be subtracted, L2 in this case). The steps for complement are explained before.
3. Take the intersection of the language L1 and $\overline{L2}$

:*Let A and B be DFA's whose languages are L and M, respectively. υConstruct C, the product automaton of A and B. υMake the final states of C be the pairs where A-state is final but B-state is not*



No incoming edges for final state (B,D) So result is empty language

8. Homomorphism

Homomorphism is allowing a string for each input symbol of a language.

For a regular language, L1, and the homomorphism, H, the language's alphabet is defined as H(L1)= {H(s)|where s is in L1}. This is also a regular language.

The process for homomorphism

Homomorphism is closed under regular languages. The algorithm is defined for the regular expression of the language.

- Define homomorphism as the operation on the regular expression of a regular language, L1.
- Prove that L1(H(R))=H(L1(R)).
- Assume R is such that L1=L1(R). Let R′=H(R), and resultantly H(L1)=L1(R′).

Example

Suppose that H(0)=ac and H(1)=bc.

Let a regular expression of a regular language be L1=001+0*.

Then, H(L1) is the language of acacbc+(ac)∗ .

By equating, we see that H(L1) is also a regular language

.

9. Inverse homomorphism

Consider H as the homomorphism and the regular language, L1. The alphabet of L1 is the output of the language of H. Therefore, H′(L1)={H(s)|where s is in L1} is also a regular language.

The process for inverse homomorphism

To prove inverse homomorphism, we perform the following steps:

Make a definite finite automaton, D, of L1.

Construct another definite finite automaton D′ such that it accepts H^{-1}(L1)..

The intuition behind this is that on the input, w, D′ will run on H(w) and accepts only if D does.

Example

Let Σ={a,b} and the input symbols as {0,1}.

Let the language L1=(00+1)∗ and H(a)=01 and H(b)=10.

Let the string be 1001, then H^{-1}(1001)={ba} .

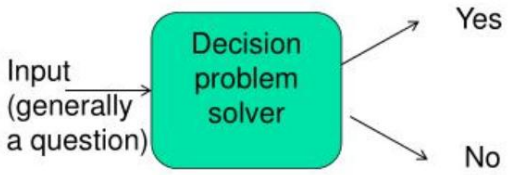
Let another string be 010110, then H^{-1}(010110)={aab}.

➢ DECISION PROPERTIES

A decision property for a class of languages is an algorithm that takes a formal description of a language (e.g., a DFA) and tells whether or not some property holds.

Example: Is language L empty?

Any "decision problem" looks like this:



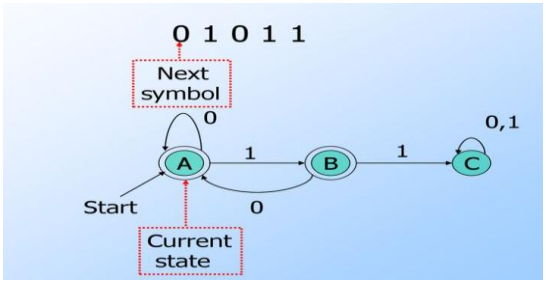
The decision properties of regular languages are membership, emptiness, finiteness, equivalence, and containment.

- **Membership**: Checks if a string is accepted by a finite automaton
- **Emptiness**: Determines if a language is empty
- **Finiteness**: Determines if a language is finite
- **Equivalence**: Determines if two languages are equivalent
- **Containment**: Determines if one language is contained within another

a) Membership Problem

• Our first decision property is the question: "is string w in regular language L?"
• Assume L is represented by a DFA A.
• Simulate the action of A on the sequence of input symbols forming w.

Testing Membership



b) Emptiness Problem

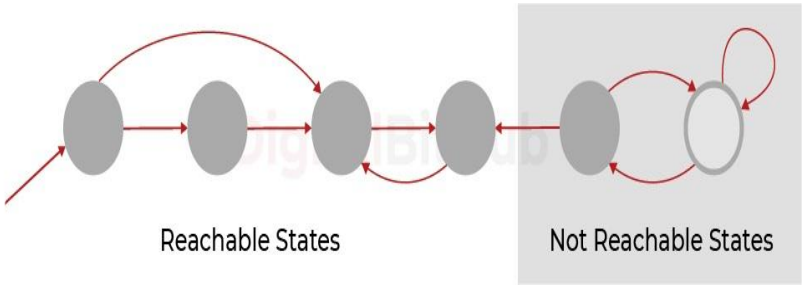
The emptiness problem in Finite Automata is to decide whether the given automata are accepting empty language or not.

There is an Algorithm available to check the emptiness of Finite Automata.

- **Step 1:** Select those states which are not reachable from the initial state, delete those states and transitions. Using DFS or BFS.
- **Step 2:** In the remaining Finite Automata, if at least one final state then it is accepting non-empty language otherwise empty language.

Conclusion: The emptiness problem of finite automata is decidable and decision problem.

Example: Consider a Finite Automata



Given FA is accepting empty language. There is no final state in reachable states.

c) Infiniteness Problem

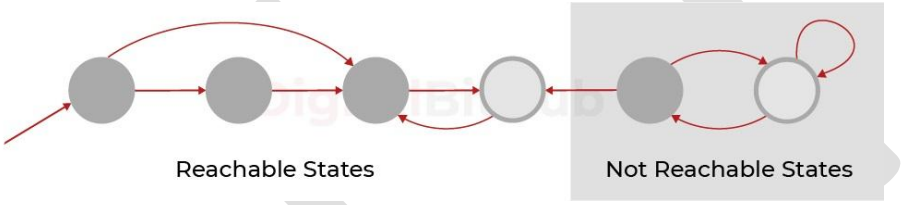
The finiteness problem in Finite Automata is to decide whether the given automata is accepting finite language or infinite language.

There is an Algorithm available to check the finiteness of Finite Automata.

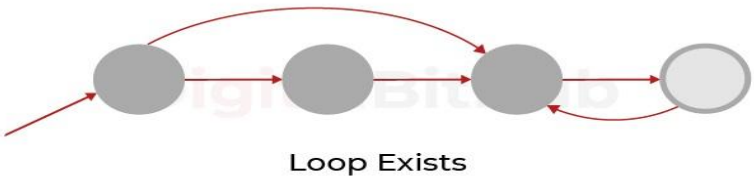
- **Step 1:** Select those states which are not reachable from the initial state, delete those states and transitions. Using DFS or BFS.
- **Step 2:** In the remaining Finite Automata, if any state from which we cannot reach the final state, delete those states and transitions.
- **Step 3:** In the remaining Finite Automata, if at least one cycle is available then it is accepting infinite language otherwise finite language.

Conclusion: The finiteness problem of finite automata is decidable and decision problem.

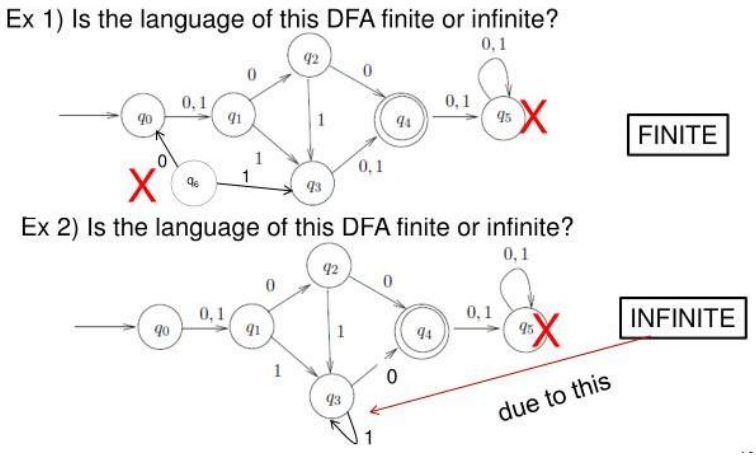
Example: Consider a Finite Automata



After applying the first 2 steps the resultant Finite Automata is



Loops exist in the resultant finite automata, hence accepting infinite language.



d) Equivalence Problem

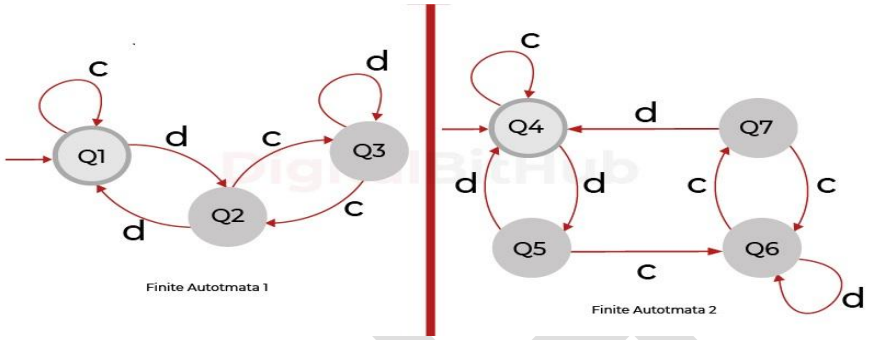
The equivalence problem in Finite Automata is to decide whether the given two automata are accepting the same language or not.

There is an Algorithm available to check the equivalence of given two Finite Automata.

- **Step 1:** Create a transition table for given FAs.
- **Step 2:** If any entry of the table contains the final state of one automaton and the non-final state of other automata then the given FAs are not accepting the same language.

Conclusion**:** The equivalence problem of finite automata is decidable and decision problem.

Example**:** Consider the following two Finite Automata



➢ **EQUIVALENCE OF 2 FINITE STATE MACHINES**

Any Two Automaton is said to be equivalent if both accept exactly the same set of input strings.

Two Automaton are equivalent if they satisfy the following conditions :

1. If FSMs are NFAs / ε-NFAs, convert them to DFAs first.

The initial and final states of both the automata must be same.

2. Every pair of states chosen is from a different automaton only. First pair is initial states of both the automata

3. While combining the states with the input alphabets, the pair results must be either both final states or intermediate states.(i.e both should lie either in the final state or in the non-final state).

4. If the resultant pair has different types of states, then it will be non-equivalent. (i.e. One lies in the final state and the other lies in the intermediate state). Otherwise, take resultant new pair and add it to the list if it is not already present , and repeat step 3, 4

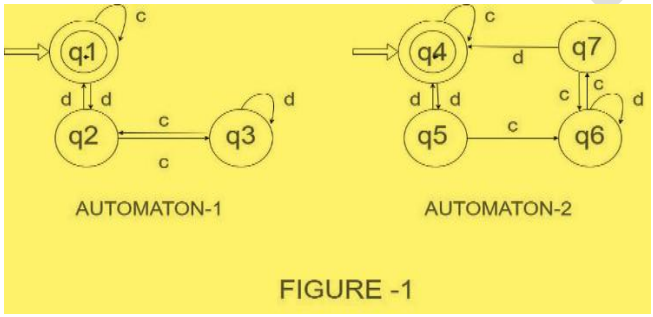


FIGURE -1

Solution –

Step 1 – Since the initial and final states of both the automaton are the same, so it verifies.

Step 2 – Check for each state by making a table of states with respect to input alphabets.

States	c	d
(q1,q4)	(q1,q4)=>(F.S,F.S)	(q2,q5)=>(I.S,I.S)
(q2,q5)	(q3,q6)=>(I.S,I.S)	(q1,q4)=>(F.S,F.S)
(q3,q6)	(q2,q7)=>(I.S,I.S)	(q3,q6)=>(I.S,I.S)
(q2,q7)	(q3,q6)=>(I.S,I.S)	(q1,q4)=>(F.S,F.S)

Here , F.S represents →Final State.
I.S represents → Intermediate State (Non-Final State) .
Step 3 – For every pair of states, the resultant states lie either in F.S or in I.S as a combination.
Conclusion – Both the Automaton are equivalent.
Note : If the resultant pair of states has a different combination of states(i.e. either (F.S, I.S) or (I.S, F.S), then both the automata are said to be non-equivalent.

➢ MINIMZATION OF FINITE AUTOMATA

Minimization of DFA means reducing the number of states from given FA. Thus, we get the FSM(finite state machine) with redundant states after minimizing the FSM.
2 approaches are there

1. Equivalence method or Partition method
2. Table filling method or myhill-nerode method

Method 1 : Equivalence method or Partition method

We have to follow the various steps to minimize the DFA. These are as follows:
Step **1:**

Eliminate all the dead states and inaccessible states from the given DFA (if any).
All those non-final states which transit to itself for all input symbols in ∑ are called as dead states.
All those states which can never be reached from the initial state are called as inaccessible states.

Step **2:**

- Draw a state transition table for the given DFA.
- Transition table shows the transition of all states on all input symbols in Σ.

Step **3**

Now, start applying equivalence theorem.

- Take a counter variable k and initialize it with value 0.
- Divide Q (set of states) into two sets such that one set contains all the non-final states and other set contains all the final states.
- This partition is called P_0.

Step **4:**

- Increment k by 1.
- Find P_k by partitioning the different sets of P_{k-1} .
- In each set of P_{k-1} , consider all the possible pair of states within each set and if the two states are distinguishable, partition the set into different sets in P_k.

Two states q1 and q2 are distinguishable in partition Pk for any input symbol 'a', if δ (q1, a) and δ (q2, a) are in different sets in partition Pk-1.

Step 5:

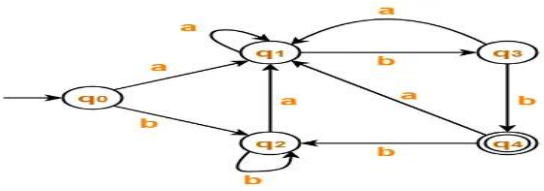
- Repeat step-04 until no change in partition occurs.
- In other words, when you find $P_k = P_{k-1}$, stop.

Step-**06:**

- All those states which belong to the same set are equivalent.
- The equivalent states are merged to form a single state in the minimal DFA.

Number of states in Minimal DFA = Number of sets in P_k

Example : Minimize given DFA



Step1 : The given DFA contains no dead states and inaccessible states.

Step2 : Transition table for DFA

	A	**B**
→q0	q1	q2
q1	q1	q3
q2	q1	q2
q3	q1	q4
***q4**	q1	q2

Step3 Now using Equivalence Theorem, we have-

$P_0 = \{ q_0 , q_1 , q_2 , q_3 \} \{ q_4 \}$

$P_1 = \{ q_0 , q_1 , q_2 \} \{ q_3 \} \{ q_4 \}$

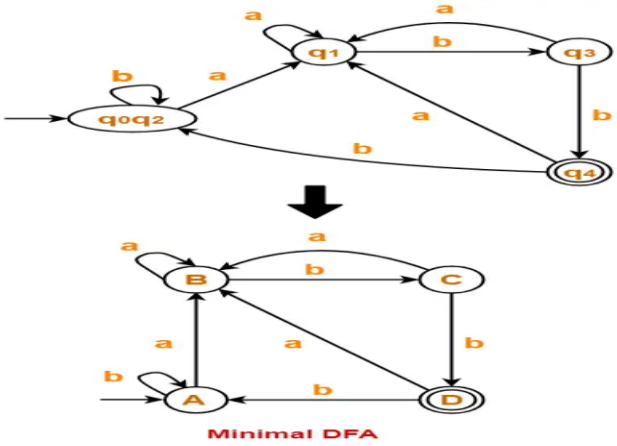
$P_2 = \{ q_0 , q_2 \} \{ q_1 \} \{ q_3 \} \{ q_4 \}$

$P_3 = \{ q_0 , q_2 \} \{ q_1 \} \{ q_3 \} \{ q_4 \}$

Since $P_3 = P_2$, so we stop.

From P_3, we infer that states q_0 and q_2 are equivalent and can be merged together.

So, Our minimal DFA is-

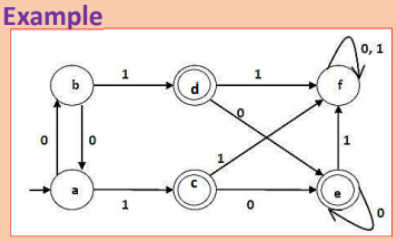


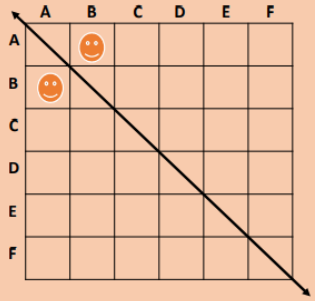
Method 2 : Table filling method or myhill-nerode method

Steps for the Minimization of DFA :

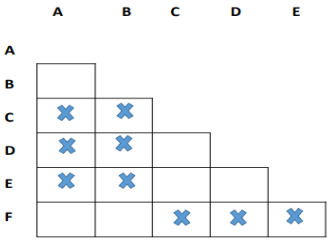
1. Create the pairs of all the states involved in the given DFA.
2. Mark all the pairs (Q_a,Q_b) such a that Q_a is Final state and Qb is Non-Final State.
3. If there is any unmarked pair (Qa, Qb) such a that δ(Qa, x) and δ(Q_b, x) is marked, then mark (Qa, Qb). Here x is a input symbol. Repeat this step until no more marking can be made.
4. Combine all the unmarked pairs and make them a single state in the minimized DFA.

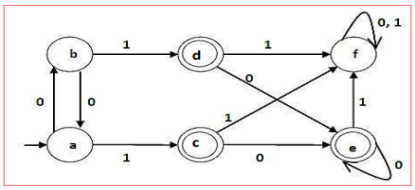
Example





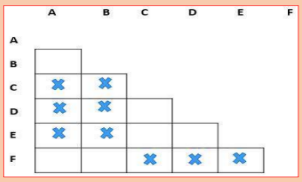
Step 1: Draw a table for all pairs of states (P, Q)



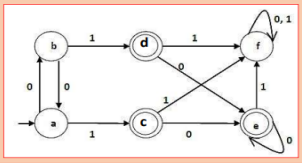


Step 2: Mark all pairs where P ∈ F and Q ∉ F (One state should be final and other should be non final state)

Step 3

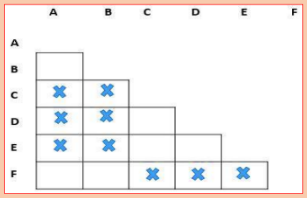


(B,A) - δ (B, 0) - A
δ (A, 0) – B unmarked
δ (B, 1) – D
δ (A, 1) – C unmarked

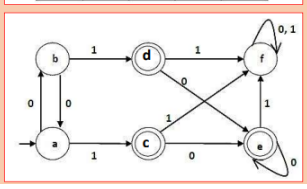


(D,C) - δ (D, 0) –E
δ (C, 0) – E no such column
δ (D, 1) – F
δ (C, 1) – F no such column

Step 3 continuation

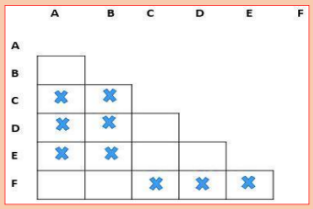


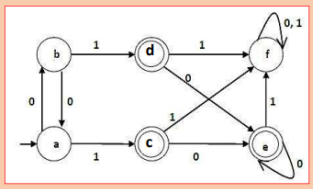
(E,C) - δ (E, 0) - E
δ (C, 0) – E no such column
δ (E, 1) – F
δ (C, 1) – F no such column



(E,D) - δ (E, 0) –E
δ (D, 0) – E no such column
δ (E, 1) – F
δ (D, 1) – F no such column

Step 3 continuation





(F, A) - δ (F, 0) - F

δ (A, 0) – B unmarked

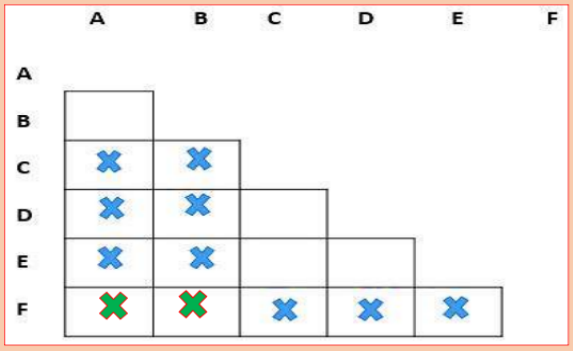
δ (F, 1) – F

δ (A, 1) – C marked. So we should mark FA

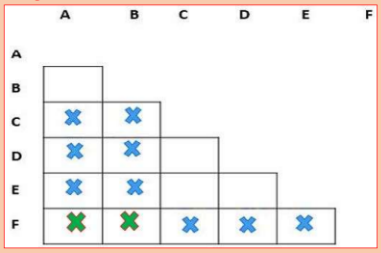
(F, B) - δ (F, 0) –F

δ (B, 0) –A FA is marked .so we should mark FB also.

Final table

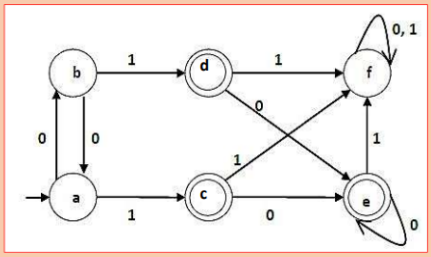


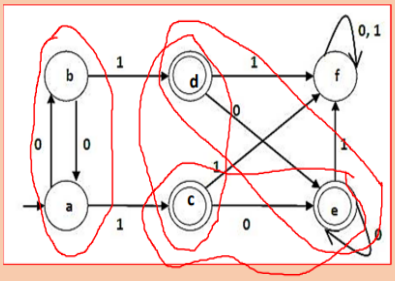
Step 4



Combine the unmarked pairs

(A,B), (D,C), (E,C), (E,D)





Minimal DFA

